Enhanced Network Attack Detection Using a TCN–BiLSTM– Transformer Framework

A Project report submitted in the partial fulfillment of the Requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

D. Anji Reddy (22471A1262)

D. Sri Saran (22471A1212)

M. Karthik (22471A1229)

Under the esteemed guidance of

Mr. N. B. S. Vijay Kumar M.Tech.,(Ph.D).

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

NARASARAOPETA ENGINEERING COLLEGE: NARASARAOPET

(AUTONOMOUS)

Accredited by NAAC with A+ Grade, ISO-9001:2015 certified, Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada

KOTAPPAKONDA ROAD, YELAMANDA VILLAGE, NARASARAOPETA-522601

2025-2026

NARASARAOPETA ENGINEERING COLLEGE: NARASARAOPET
(AUTONOMOUS)
DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project work that is entitled "Enhanced Network Attack Detection Using a TCN–BiLSTM–Transformer Framework" is a bonafide word done by the team D. Anji Reddy (22471A1262), D. Sri Saran (22471A1212), M. Karthik (22471A1229) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of INFORMATION TECHNOLOGY during the academic year 2025-2026.

PROJECT GUIDE

Mr. N. B. S. Vijay Kumar M.Tech., (Ph.D).
Assistant Professor

PROJECT CO ORDINATOR

Dr. Sk. Mohammad Jany M.Tech., Ph.D.
Associate Professor

HEAD OF THE DEPARTMENT

Dr. B. Jhansi Vazram B.Tech., M.Tech., Ph.D.
Professor& HOD

EXTERNAL EXAMINER

Viva Voce Date……

DECLARATION

We hereby declare that the Major project work entitled **"Enhanced Network Attack Detection Using a TCN–BiLSTM–Transformer Framework"** is an outcome of our efforts which has been duly acknowledged. We also declare that it has not been previously or currently submitted for any other institutions or universities. This project is done under the guidance of **Mr. N. B. S. Vijay Kumar M.Tech., (Ph.D.), Assistant Professor** Department of INFORMATION TECHNOLOGY, **NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS),** is submitted in the partial fulfillment of the requirements for the award of the degree in INFORMATION TECHNOLOGY.

Submitted by:

D. Anji Reddy (22471A1262)
D. Sri Saran (22471A1212)
M. Karthik (22471A1229)

ACKNOWLEDGEMENTS

We wish to express my thanks to carious personalities who are responsible for the completion of the project. We are extremely thankful to our beloved chairman **Sri M. V. Koteswara Rao** B.Sc., who took keen interest in us in every effort throughout this course. We owe out sincere gratitude to our beloved principal **Dr. S. Venkateswarlu** M.Tech., Ph.D. for showing his kind attention and valuable guidance throughout the course.

We express our deep felt gratitude towards **Dr. B. Jhansi Vazram** M.Tech., Ph.D., **Head of the Department** of Information Technology, for her continuous support and encouragement and also to our guide **Mr. N. B. S. Vijay Kumar** M.Tech.,(ph.D) **Assistant Professor**, Department of Information Technology whose valuable guidance and unstinting encouragement enable us to accomplish our project successfully in time.

We extend our sincere thanks towards **Dr. SK. Mohammed Jany** M.Tech., Ph.D., **Associate Professor** & Project coordinator for extending his encouragement. Their profound knowledge and willingness have been a constant source of inspiration for us throughout this project work.

We extend our sincere thanks to all other teaching and non-teaching faculty of the department for their cooperation and encouragement during our B.Tech degree. We have no words to acknowledge the warm affection, constant inspiration and encouragement that we received from our parents.We affectionately acknowledge the encouragement received from our friends and those who involved in giving valuable suggestions had clarifying out doubts which had really helped us in successfully completion of our project.

Submitted by:

D. Anji Reddy (22471A1262)

D. Sri Saran (22471A1212)

M. Karthik (22471A1229)

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF INFORMATION TECHNOLOGY

INSTITUTE VISION AND MISSION

INSTITUTE VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community.

INSTITUTE MISSION

M1: Provide the best class infra-structure to explore the field of engineering and research.

M2: Build a passionate and a determined team of faculty with student centric teaching, imbibing experiential, innovative skills.

M3: Imbibe lifelong learning skills, entrepreneurial skills and ethical values in students for addressing societal problems.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF INFORMATION TECHNOLOGY

DEPARTMENT VISION AND MISSION

DEPARTMENT VISION

To transform into a research and technological hub to develop prominent IT professionals to serve the needs of industry and society.

DEPARTMENT MISSION

The department of Information Technology is committed to

M1: Induce preliminary and contemporary IT principles of the industry among the students

M2: Develop strong force of students to solve the real time problems of the IT industry.

M3: Incubate the students with emerging entrepreneur intelligence.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Ability to analyze and develop computer programs in the areas related to Algorithms, system software, application software, web design, big data analytics, database design and networking for efficient design of computer-based systems of varying complexity.

PSO2: Design, Implement and evaluate a computer based system to meet desired needs.

PSO3: Develop IT application services with the help of different current engineering tools.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

The graduates of the programmer are able to:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Information Technology problems.

PEO2: Use various software tools and technologies to solve problems related to academia, industry and society.

PEO3: Work with ethical and moral values in the multi disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in software industry.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAM OUTCOMES (POs)

PO1: Engineering knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in respectively to develop to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct investigations of complex problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: Engineering tool usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project management and finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-long learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF INFORMATIONTECHNOLOGY

PROJECT WORK - COURSE OUTCOMES (Cos)

CO421.1: Analyze the System of Examinations and identify the problem.

CO421.2: Identify and classify the requirements.

CO421.3: Review the Related Literature**.**

CO421.4: Design and Modularize the project.

CO421.5: Construct, Integrate, Test and Implement the Project.

CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes–Program Outcome correlation

	PO1	PO2	PO3	PO4	P05	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1	2	3										2		
C421.2			2		3							2		
C421.3				2		2	3					2		
C421.4			2			1	2					3	2	
C421.5					3	3	2	3	2	2	1	3	2	1
C421.6								3	2	1		2	3	

Note: The values in the above table represent the level of correlation between Cos and POs:

1. Low level
2. Medium level
3. High level

Project work mapping with various courses of Curriculum with Attained POs:

Name of the course from which principles are applied in this project	Description of the device	Attained PO
R20CC2204, R20CC22L3	Gathering the requirements and defining the problem, plan to develop	PO1, PO3
R20CC42, R20CC2204, R20CC22L3	Each and every requirement is critically analyzed, the process model is identified and divided into	PO2, PO3
R20CC421, R20CC2204, R20CC22L3	Logical design is done by using the unified modeling language which involves individual team work	PO3, PO5, PO8
R20CC421, R20CC2204, R20CC22L3	Each and every module is tested, integrated, and evaluated in our project	PO1, PO5
R20CC421, R20CC2204, R20CC22L3	Documentation is done by all our three members in the form of a group	PO9,PO10
R20CC421, R20CC2204, R20CC22L3	Each and every phase of the work in group is presented periodically	PO9,PO10
R20C2202, R20C2203, R20C1206, R20C3204, R20C4110	Implementation is done and the project will be handled by the	PO4
R20CC32SC4	The physical design of the proposed intrusion detection system includes both hardware and software components required for efficient data processing, model training, and deployment.	PO5, PO6

ABSTRACT

The risk of network security is continuously growing in scope and complexity and therefore in need of intrusion detection systems, which would be capable of deriving meaningfulness out of types and dynamically changing traffic patterns. Previous methodologies have created shortcomings in several key areas which include, but are not limited to, subjected bias, spatiotemporal relationships, and the pattern recognition required for the analysis of large amounts of interconnected data. To tackle such issues, an exemplary work has presented in this case an architecture which includes the fusion of Convolutional Networks, Bidirectional Long ShortTerm models, and Transformer encoders. The integrated frameworks explain network traffic in the short-term, the long- term, and incorporate the global dependencies.

The models are trained and tested in NSL- KDD and UNS-NB15 datasets, in the course of data preprocessing includes Min–Max normalization, one-hot encoding of categorical features, BRFE-based feature selection, sliding window temporal segmentation, and SMOTE oversampling to address severe class imbalance. Experimental evaluation on NSL-KDD and UNSW-NB15 datasets demonstrates that the proposed TCN–BiLSTM– Transformer model achieves superior accuracy, precision, and F1-score compared to Random Forest, CNN, CNN-LSTM, and CNN-BiLSTM models.The results indicate profound temporal detail extraction will considerable boost the efficiency and effectiveness of intrusion detection systems, and facilitate the detection of a wide variety of attack forms.

KEYWORDS: Network intrusion detection, temporality feature extraction, TCN, BiLSTM, Transformer encoder, network traffic analysis, class imbalance, feature selection, sliding-window, cybersecurity.

INDEX

LIST OF FIGURES

1. INTRODUCTION

1.1 Introduction

The rapid expansion of digital communication networks has significantly increased the volume, complexity, and diversity of network traffic. As organizations increasingly rely on interconnected systems, cloud platforms, and IoT devices, the risk of sophisticated cyberattacks has grown correspondingly. Traditional security mechanisms such as firewalls and signature-based intrusion detection systems are often inadequate in detecting modern, stealthy, and multi-stage attacks. These evolving threats demand intelligent intrusion detection systems (IDS) capable of analyzing complex traffic patterns and identifying both known and unknown attack behaviors in real time.

Conventional machine learning approaches for intrusion detection typically focus on static packet-level features and fail to effectively capture temporal dependencies within network traffic. However, cyberattacks often unfold as sequences of coordinated actions over time rather than isolated events. Moreover, real-world intrusion datasets such as NSL-KDD and UNSW-NB15 are highly imbalanced, where rare but critical attack categories are underrepresented. This imbalance reduces the detection sensitivity of many traditional models and increases false negatives for minority attack classes, posing serious security risks.

To address these limitations, this project proposes a hybrid deep learning architecture that integrates Temporal Convolutional Networks (TCN), Bidirectional Long Short-Term Memory (BiLSTM), and Transformer encoders. The TCN component efficiently captures short-term temporal patterns and local feature dependencies, while the BiLSTM layer models long-range sequential relationships in both forward and backward directions. The Transformer encoder further enhances the system by leveraging self-attention mechanisms to learn global contextual dependencies across entire traffic sequences. By combining these complementary strengths, the proposed framework provides multi-level temporal learning and improved feature representation.

In addition to architectural innovation, the system incorporates a comprehensive preprocessing and optimization pipeline. Techniques such as Min–Max normalization, one- hot encoding of categorical variables, Boosted Recursive Feature Elimination (BRFE), sliding-window temporal segmentation, and SMOTE-based oversampling are employed to enhance feature quality and address severe class imbalance. This structured data preparation ensures improved generalization capability, reduced overfitting, and enhanced sensitivity to rare attack categories.

Experimental evaluation on benchmark datasets demonstrates that the proposed TCN– BiLSTM–Transformer model achieves superior accuracy, precision, and F1-score compared to baseline models such as Random Forest, CNN, CNN-LSTM, and CNN- BiLSTM. The results confirm that integrating

short-term, long-term, and global dependency modeling significantly improves intrusion detection performance. Overall, this work presents a robust, scalable, and reliable framework for modern network intrusion detection systems capable of operating effectively in dynamic and high-dimensional network environments.

The proposed framework addresses these issues through a multi-stage processing pipeline. First, data preprocessing ensures consistency through cleaning, normalization, and categorical encoding. Next, Boosted Recursive Feature Elimination (BRFE) selects the most informative attributes, reducing dimensionality while preserving discriminative power. Sliding-window segmentation transforms traffic records into temporal sequences, enabling sequential learning. SMOTE-based oversampling is applied to generate synthetic minority samples, ensuring that the model learns meaningful representations for underrepresented attack classes. These preprocessing techniques collectively strengthen the model's stability and generalization.

The hybrid TCN–BiLSTM–Transformer architecture is designed to capture traffic behavior at different analytical levels. The TCN module extracts short-range temporal features using dilated causal convolutions, enabling parallel computation and efficient receptive field expansion. The BiLSTM layer captures bidirectional dependencies, learning how past and future traffic patterns influence current behavior. The Transformer encoder introduces a self-attention mechanism that assigns dynamic importance weights to different time steps, effectively modeling global interactions within traffic sequences. The fusion of these components results in a comprehensive representation of network activity, leading to enhanced classification accuracy and reduced false alarms.

In summary, this work contributes to the advancement of intelligent intrusion detection by integrating multi-level temporal learning, optimized preprocessing, and attention-based global modeling within a unified deep learning framework. The experimental findings validate that combining short-term feature extraction, long-term dependency modeling, and global contextual learning significantly improves detection performance compared to traditional and single-model approaches. The proposed system demonstrates strong potential for real-world deployment in enterprise networks, cloud environments, and critical infrastructure systems where accurate and timely threat detection is essential.

Beyond the increasing sophistication of cyber threats, the dynamic nature of modern network environments further complicates intrusion detection. With the rapid adoption of technologies such as edge computing, software-defined networking (SDN), and 5G infrastructures, network behavior has become highly heterogeneous and distributed. This evolution introduces new vulnerabilities and expands the attack surface, making it more difficult to identify malicious activities using static or rule-based approaches. As a result, intrusion detection systems must not only be accurate but also adaptive, scalable, and capable of processing high-velocity data streams in real time.

Another critical challenge lies in the interpretability and reliability of detection models. While deep

learning techniques have demonstrated high performance, their black-box nature often limits transparency, which is essential for security analysts to understand and respond to detected threats. Therefore, there is a growing need for models that balance predictive performance with explainability, enabling trust and practical deployment in real- world scenarios. Additionally, reducing false positives is equally important, as excessive alerts can overwhelm security teams and lead to alert fatigue, potentially causing critical threats to be overlooked.

Furthermore, the integration of intelligent intrusion detection systems with automated response mechanisms is becoming increasingly important. Modern cybersecurity frameworks aim not only to detect attacks but also to respond proactively through automated mitigation strategies. This requires detection models to be robust, fast, and capable of continuous learning to adapt to emerging attack patterns. In this context, hybrid deep learning approaches offer a promising direction by combining multiple learning paradigms to achieve both high detection accuracy and adaptability.

Modern computer networks have become highly complex due to the rapid growth of digital communication, cloud computing, and interconnected systems. This expansion has increased the exposure of networks to various cyber threats, including malware, phishing, and distributed attacks. Traditional security systems are no longer sufficient to handle these advanced threats because attackers continuously evolve their techniques. Therefore, there is a strong need for intelligent systems that can monitor network traffic and detect malicious activities efficiently.

Intrusion Detection Systems (IDS) play a vital role in identifying unauthorized access and abnormal behavior within a network. However, many existing systems rely on static rules or simple models that fail to adapt to dynamic attack patterns. With the advancement of machine learning and deep learning, IDS can now analyze large volumes of data and identify hidden patterns in network traffic. These intelligent approaches help improve detection accuracy and reduce false alarms.

This project focuses on developing an advanced intrusion detection framework using a hybrid deep learning model. By combining multiple learning techniques, the system aims to capture different characteristics of network behavior, including short-term variations, long-term dependencies, and global relationships. The goal is to design a reliable and efficient system that can detect both known and unknown attacks in real-time environments.

In addition, the project incorporates advanced preprocessing techniques such as normalization, feature selection, and data balancing to improve model performance. These steps ensure that the input data is clean, structured, and suitable for training. Overall, this work aims to develop a robust, scalable, and efficient intrusion detection system capable of operating in real-world environments.

Overall, the advancement of intrusion detection systems is moving toward intelligent, adaptive, and context-aware solutions that can operate effectively in complex and evolving network ecosystems. This shift highlights the importance of developing comprehensive frameworks that integrate advanced learning

techniques, efficient data processing strategies, and practical deployment considerations.

1.1 Existing System

The existing intrusion detection systems (IDS) primarily rely on traditional machine learning techniques and standalone deep learning models to classify network traffic as normal or malicious. Conventional IDS approaches can be broadly categorized into signature-based and anomaly-based systems. Signature-based systems detect attacks by matching traffic patterns against a predefined database of known attack signatures. While effective for previously identified threats, these systems fail to detect zero-day attacks and new intrusion patterns. On the other hand, anomaly-based systems attempt to model normal network behavior and flag deviations as potential intrusions, but they often suffer from high false positive rates.

With the advancement of deep learning, models such as Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Bidirectional LSTM (BiLSTM) have been applied to intrusion detection. CNN-based approaches are effective at capturing local spatial patterns in traffic features but lack the ability to model long-term temporal dependencies. LSTM-based models can learn sequential relationships; however, they may suffer from vanishing gradient issues and computational inefficiency when processing long sequences. Although BiLSTM improves contextual understanding by learning bidirectional dependencies, it still cannot efficiently model global feature interactions across entire traffic flows.

In addition to these limitations, many existing intrusion detection systems struggle with scalability and real-time processing requirements. Modern network environments generate massive volumes of high-speed data, making it challenging for traditional and standalone deep learning models to process traffic efficiently without introducing latency. Models such as LSTM and BiLSTM, while powerful in sequence learning, often require significant computational resources and longer training times, which limits their applicability in real- time intrusion detection scenarios.

Another key issue in existing systems is their heavy dependence on feature engineering. Traditional machine learning approaches rely on manually crafted features, which may not fully capture the complex and evolving nature of network traffic. Even in deep learning- based systems, insufficient feature representation can lead to suboptimal performance, especially when dealing with encrypted traffic or highly dynamic communication patterns. This dependency reduces the model's adaptability to new and unseen attack vectors.

Moreover, many existing IDS models are trained and evaluated on benchmark datasets under controlled conditions, which may not accurately reflect real-world network environments. As a result, these models often experience performance degradation when deployed in practical scenarios due to variations in traffic distribution, noise, and unseen attack behaviors. The lack of robustness and generalization remains a significant challenge in current intrusion detection research.

Class imbalance is another persistent problem in existing systems. Network intrusion datasets typically contain a large proportion of normal traffic and only a small fraction of attack instances. Most traditional and deep learning models tend to be biased toward the majority class, leading to poor detection rates for rare but critical attack types. This imbalance significantly affects the reliability of IDS in identifying sophisticated and low- frequency attacks.

Anomaly-based systems attempt to overcome this limitation by modeling normal network behavior and identifying deviations as potential intrusions. Although this approach can detect unknown attacks, it often suffers from high false positive rates. Many normal activities may be incorrectly classified as malicious, leading to unnecessary alerts and reduced system reliability. This creates challenges for security analysts who must differentiate between genuine threats and false alarms.

With the introduction of machine learning, several models such as Decision Trees, Support Vector Machines, and Random Forest have been applied to intrusion detection. These models improve classification accuracy compared to traditional systems; however, they rely heavily on manually engineered features. They treat each data instance independently and fail to capture the sequential nature of network traffic, which is crucial for detecting complex and multi-stage attacks.

Furthermore, existing standalone models often operate in isolation, focusing on either spatial or temporal feature extraction but not both simultaneously. This limited learning capability restricts their ability to capture multi-dimensional relationships within network traffic. As cyberattacks become more complex and multi-stage, the inability to model short-term patterns, long-term dependencies, and global contextual relationships together becomes a major drawback.

1.2 Proposed System

The proposed system introduces a hybrid deep learning–based intrusion detection framework designed to overcome the limitations of traditional and standalone models. The system integrates Temporal Convolutional Networks (TCN), Bidirectional Long Short- Term Memory (BiLSTM), and Transformer encoders to effectively capture short-term, long-term, and global dependencies in network traffic data. By combining these complementary architectures, the model provides a more comprehensive understanding of complex and evolving cyberattack patterns.

Fig 1.3 system utilizes benchmark intrusion detection datasets such as NSL-KDD and UNSW-NB15. Initially, the raw network traffic data undergoes preprocessing to ensure consistency and reliability. Missing or irregular values are handled appropriately to avoid bias in model training. Numerical attributes such as packet size, duration, and byte count are normalized using Min–Max normalization to bring all features into a uniform range. Categorical features such as protocol type and service are converted into numerical format using one-hot encoding. To reduce computational complexity and eliminate redundant attributes, Boosted Recursive Feature Elimination (BRFE) is applied. This method ranks

features based on importance scores derived from gradient boosting techniques and iteratively removes less significant attributes. The selected optimal feature subset improves model efficiency, reduces overfitting, and enhances generalization capability. The model is trained using the Adam optimizer with an adaptive learning rate to ensure stable convergence. Dropout regularization is incorporated to prevent overfitting, and early stopping is used to retain the best-performing model weights. The system is evaluated using accuracy, precision, recall, F1-score, and confusion matrix analysis to ensure comprehensive performance assessment. The proposed system introduces a hybrid deep learning architecture designed to overcome the limitations of existing intrusion detection methods. It combines multiple advanced techniques to analyze network traffic from different perspectives. This integration enables the system to understand both immediate and long-term patterns in data, leading to improved detection performance.

To reduce computational complexity and eliminate redundant attributes, Boosted Recursive Feature Elimination (BRFE) is applied. This method ranks features based on importance scores derived from gradient boosting techniques and iteratively removes less significant attributes.

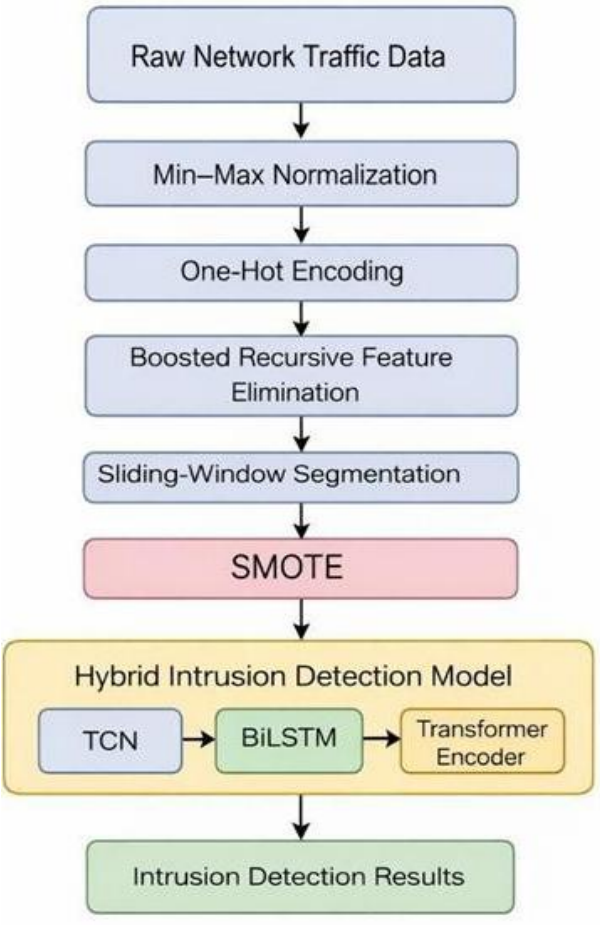

Fig 1.3: Flowchart For Proposed System

To further enhance the learning capability of the proposed framework, the architecture is designed to enable efficient information flow between its components. The TCN layer processes input sequences using dilated convolutions, allowing the model to expand its receptive field without increasing computational cost. This ensures that short-term temporal variations and local dependencies are captured effectively at an early stage. The extracted feature maps are then passed to the BiLSTM layer, where bidirectional sequence processing enables the model to understand contextual relationships from both past and future perspectives.

Subsequently, the Transformer encoder refines the learned representations by applying multi-head self-attention mechanisms. This allows the model to dynamically focus on the most relevant parts of the sequence, capturing global interactions that are often missed by sequential models alone. The attention mechanism assigns different weights to different time steps, enabling the system to prioritize critical patterns associated with intrusion behavior. This layered processing ensures that the model captures hierarchical features ranging from local patterns to global dependencies.

The proposed system introduces an advanced hybrid deep learning framework designed to overcome the limitations of existing intrusion detection systems. This system integrates three powerful architectures—Temporal Convolutional Networks (TCN), Bidirectional Long Short-Term Memory (BiLSTM), and Transformer encoders—to provide a comprehensive analysis of network traffic data. By combining these models, the system is capable of capturing short-term patterns, long-term dependencies, and global contextual relationships simultaneously.

The system is also designed with computational efficiency in mind. Parallel processing capabilities of the TCN and the attention mechanism of the Transformer help reduce training time compared to purely sequential models. This makes the framework more suitable for large-scale and real-time network environments. Additionally, the modular design of the architecture allows flexibility for future enhancements, such as integrating online learning or adaptive thresholding mechanisms.

To ensure robustness, the model incorporates batch-wise training and validation strategies that maintain stability across different data distributions. The framework is capable of handling high-dimensional input data while maintaining consistent performance. It is also adaptable to different types of network traffic, including encrypted and heterogeneous data streams, making it suitable for modern network infrastructures.

The system begins with a preprocessing stage where raw network data is cleaned, transformed, and optimized. Important features are selected to reduce complexity, and data balancing techniques are applied to ensure fair learning across all classes. These steps enhance the quality of input data and improve the overall efficiency of the model.

The core of the system is a multi-layer architecture that processes data through different components.

Each component focuses on a specific aspect of learning, such as local pattern recognition, sequential analysis, and global attention. By combining these outputs, the system achieves higher accuracy, better generalization, and reduced false alarms, making it suitable for real-world cybersecurity applications.

Finally, the proposed system emphasizes reliability in deployment by minimizing false alarms and maintaining high detection sensitivity. Its ability to generalize across diverse attack patterns ensures consistent performance even in dynamic and evolving cyber environments. This makes the framework a strong candidate for practical intrusion detection applications in enterprise networks, cloud systems, and critical infrastructure.

Additionally, the modular design of the architecture allows flexibility for future enhancements, such as integrating online learning or adaptive thresholding mechanisms.

The system begins with a data preprocessing stage, where raw network traffic data is cleaned and transformed into a suitable format. Missing values and inconsistencies are handled to ensure data quality. Numerical features are normalized to maintain uniformity, while categorical features are converted into numerical form using encoding techniques. Feature selection methods are applied to remove redundant and irrelevant attributes, reducing complexity and improving model efficiency.

To address the issue of class imbalance, advanced techniques such as oversampling are used to generate synthetic data for minority classes. This ensures that the model learns effectively from all types of data, including rare attack categories. Additionally, sliding window techniques are applied to convert traffic records into sequences, enabling the model to analyze temporal patterns.

The core of the system consists of the hybrid architecture. The TCN component captures short-term dependencies using dilated convolutions, allowing efficient parallel processing. The BiLSTM layer analyzes sequential data in both forward and backward directions, capturing long-range dependencies. The Transformer encoder enhances the model by applying attention mechanisms to identify important features across the entire sequence.

The outputs from these components are combined to form a unified representation, which is then used for classification. This multi-level analysis significantly improves detection accuracy and reduces false positives. The system is designed to be scalable and efficient, making it suitable for real-time applications in modern network environments.

To ensure robustness, the model incorporates batch-wise training and validation strategies that maintain stability across different data distributions. The framework is capable of handling high-dimensional input data while maintaining consistent performance. It is also adaptable to different types of network traffic, including encrypted and heterogeneous data streams, making it suitable for modern network infrastructures.

1.3 System Requirements

1.3.1 Hardware Requirements

- Processor : Intel Core i5
- Cache Memory : 4MB
- Hard Disk : 30GB or more
- RAM : 1GB or more

1.3.2 Software Requirements

- Operating System : Windows 10
- Coding Language : Python
- Python Distribution : Anaconda, Flask
- Browser : Any Latest Browser Like Chrome

The system requirements for this application include both hardware and software specifications necessary to ensure smooth development, training, and deployment of the intrusion detection system. Since the proposed model involves deep learning techniques and data preprocessing operations, the system must have adequate computational capability and storage capacity to handle large network traffic datasets and model training processes efficiently.

For hardware configuration, the recommended processor is an Intel Core i5 or higher, as it provides sufficient computational power to execute data preprocessing, feature selection, and model training tasks. A minimum of 4MB cache memory is suggested to improve processing efficiency and reduce latency during execution. The system should have at least 1GB RAM; however, higher RAM capacity is preferable for better performance, especially when handling large datasets such as NSL-KDD and UNSW-NB15.

The software requirements include a compatible operating system, development environment, and necessary tools for implementation and deployment. The application is designed to run on Windows 10 or later versions, providing a stable and user-friendly platform for development. The coding language used for implementation is Python, due to its extensive support for machine learning and deep learning frameworks. Python distribution platforms such as Anaconda are recommended, as they simplify package management and environment configuration. Frameworks such as Flask are utilized for building the web-based interface and deploying the intrusion detection system.

2. LITERATURE SURVEY

Intrusion Detection Systems (IDS) have evolved significantly over the past two decades due to the rapid growth of networked systems and increasing cybersecurity threats. Early IDS approaches were primarily signature-based, where known attack patterns were stored in a database and matched against incoming traffic. Although these systems were effective in detecting previously identified threats, they failed to recognize zero-day attacks and novel intrusion patterns. This limitation led researchers to explore anomaly-based detection techniques that model normal network behavior and flag deviations as potential threats.

Traditional machine learning methods such as Decision Trees, k-Nearest Neighbors (k- NN), Naïve Bayes, and Support Vector Machines (SVM) were widely applied to intrusion detection tasks. These models relied heavily on handcrafted features extracted from network traffic data. Random Forest, in particular, showed improved robustness due to ensemble learning and feature importance ranking. However, these approaches treated traffic records independently and lacked the ability to model sequential and temporal dependencies inherent in network traffic.

With the advancement of deep learning, researchers began applying neural network architectures to intrusion detection. Convolutional Neural Networks (CNNs) demonstrated strong performance in extracting spatial patterns from structured network traffic features. CNN-based models were particularly effective in identifying well-defined attack signatures and traffic correlations. However, CNNs are inherently limited in capturing long-term temporal dependencies since they primarily focus on local feature extraction.

To address temporal learning challenges, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were introduced. LSTM models improved intrusion detection performance by capturing sequential dependencies across traffic flows. They were capable of remembering historical information and detecting time-based attack patterns. Nevertheless, standard LSTMs suffered from high computational cost and difficulty in parallelization, especially when dealing with long sequences of network data. Bidirectional LSTM (BiLSTM) further enhanced sequential modeling by processing input data in both forward and backward directions. This allowed the model to understand contextual relationships more effectively. Several studies reported improved recall and F1- scores using BiLSTM compared to unidirectional LSTM. Despite these improvements, BiLSTM models still faced challenges in capturing global feature interactions across entire traffic sequences.

Recent advancements introduced attention mechanisms to improve deep learning-based IDS performance. The Transformer architecture, based on self-attention mechanisms, demonstrated superior capability in modeling long-range dependencies without relying on recurrent structures. Transformers assign dynamic weights to different time steps, enabling the model to focus on the most relevant parts of

the sequence. Researchers found that attention-based models improved classification accuracy and reduced false positives in complex network environments.

Hybrid deep learning architectures have gained popularity due to their ability to combine complementary strengths of different models. CNN-LSTM and CNN-BiLSTM hybrid models were proposed to extract both spatial and temporal features simultaneously. These models achieved better performance than standalone CNN or LSTM models. However, most hybrid architectures still lacked global contextual modeling and efficient long- distance dependency learning.

Another significant research focus has been feature optimization. High-dimensional datasets such as NSL-KDD and UNSW-NB15 contain redundant and irrelevant features that increase computational complexity. Feature selection methods such as Recursive Feature Elimination (RFE), Principal Component Analysis (PCA), and XGBoost-based selection techniques were employed to reduce dimensionality. Optimized feature subsets improved classification accuracy while reducing training time.

Class imbalance remains one of the major challenges in intrusion detection research. Many datasets exhibit skewed distributions where minority attack categories such as User-to- Root (U2R) and Remote-to-Local (R2L) are significantly underrepresented. Researchers applied oversampling techniques such as SMOTE (Synthetic Minority Oversampling Technique) to generate synthetic samples for minority classes. Balanced training improved recall for rare attacks and enhanced model reliability.

Temporal modeling techniques such as sliding-window segmentation have also been explored to convert raw traffic records into time-series sequences. This approach enables models to learn attack progression patterns instead of isolated packet behaviors. Studies show that sequence-based modeling significantly improves detection of stealthy and multi- stage attacks compared to static classification methods.

Recent works emphasize the importance of combining multi-level temporal learning with attention-based global modeling. Hybrid architectures integrating TCN, LSTM variants, and Transformer encoders have demonstrated promising results in benchmark datasets.

Temporal Convolutional Networks (TCN) offer efficient parallel computation and larger receptive fields using dilated convolutions, making them suitable for short-term dependency modeling. When combined with BiLSTM and Transformer, they provide comprehensive feature extraction at multiple levels.

In summary, existing literature highlights the transition from traditional machine learning methods to advanced hybrid deep learning architectures for intrusion detection. While CNN, LSTM, BiLSTM, and Transformer-based models have individually shown promising results, no single model fully captures short-term, long-term, and global dependencies simultaneously. This research gap motivates the development of integrated frameworks such as the proposed TCN–BiLSTM–Transformer model, which aims to enhance detection accuracy, improve minority attack recognition, and provide a robust solution

for modern cybersecurity challenges.

Intrusion detection in networks has seen gradual improvement. many of the researchers explore the integration of various. Addressing the challenges of contemporary networks. While systematic cyberattacks take place in a short period of time, the signature-based and statistical approaches of an ID system are no longer enough, resulting in the use of an automated, adaptive, and intelligent system that can learn from the diverse and dynamically evolving patterns of an attack. More recent research focuses on the cognitive. Incorporation of the ability to forecast time, class imbalance, and lack of robustness against various challenges.Equipped with the above, the hybrid system TCN-BiLSTM- Transformers model established by Jin locally integrated temporal features and an outstanding architecture with the aforementioned improved achievements, and global attention on various datasets exceeded performance [1].

Early works on intrusion detection based on deep learning showed that hybrid networks can execute traditional machine learning techniques. Hassan et al. emphasized the effectiveness of combining CNN and LSTM layers, and the results provide a significant improvement in terms of accuracy using multi stage temporal modeling techniques [2]. Accordingly, Liu and Patras analyzed Bi-directional LSTM variants, NoiseInvar ants utilized in examining bidirectional dependencies in network flows, displaying a heightened sensitivity towards Sequential Attack Behavior [3]. Some research covered convolution- based architectures “for traffic analysis”. Guler ¨ and Alpay studied RNN, LSTM, and the GRU models under high-dimensional network conditions, stressing the need to consider architectures that balance accuracy and computational efficiency [4]. Other research efforts combined CNN-LSTM networks for improvement in research efforts feature extraction

from packet traces, proof that Hybrid models can clearly outperform CNN models alone approaches[5].

Feature optimization as a key area of focus as well. Kasongo introduced the approach of utilizing XGBoost-LSTM to filter redundant features, which enhances the detection of minority attacks classes in imbalanced datasets [6]. Shi et al. have further this approach through using a structure of Transformer-BiLSTM to grasp both long-distance and world level connections within traffic sequences, demonstrating that attention-based mechanisms aid to more reliable classifications [7]. There have also been recent studies that have focused on the performance of IDS under realistic network environments. AbdulRaheem et al. assessed intrusion detection in SDN-based architectures using machine learning methods, which show that optimized Preprocessing and adaptive learning techniques play an important role in deployment in high-speed networks [8]. Wang et al. continued research on transfer learning techniques to improve detection in cyber-physical systems, focusing on the importance of cross-domain adaptability [9]. Full fledged deep learning models are steadily emerge, and works like Su et al. demonstrating the end-to-end models applied to NSL-KDD can provide substantial gains and when paired with proper preprocessing,

normalization, and batch level feature extraction [10].

Vaswani et al. introduced the Transformer architecture, which uses self-attention mechanisms to model global dependencies in sequential data without relying on recurrent structures. Their work demonstrated significant improvements in sequence modeling tasks and has since been widely adopted in various domains, including network intrusion detection [11]. Recent research has also focused on improving the generalization capability of intrusion detection models across different network environments. Many IDS models perform well on benchmark datasets but fail when deployed in real-world scenarios due to distribution shifts and unseen traffic patterns. To address this, transfer learning and domain adaptation techniques have been introduced. These approaches allow models trained on one dataset to adapt to another network environment with minimal retraining, improving cross-domain robustness and practical applicability. Another growing area of research involves Software-Defined Networking (SDN) and cloud-based intrusion detection systems. In SDN environments, centralized controllers provide global visibility of network flows, enabling more intelligent traffic analysis. Machine learning models integrated into SDN controllers can dynamically monitor and mitigate threats in real time. Studies have shown that deep learning–based IDS models significantly improve detection accuracy in SDN architectures compared to traditional rule-based systems.

The rise of Internet of Things (IoT) networks has introduced new challenges for intrusion detection due to resource constraints and heterogeneous device communication. IoT traffic often consists of lightweight, irregular, and device-specific communication patterns. Researchers have proposed lightweight deep learning models and edge-based IDS deployment strategies to ensure real-time detection without overwhelming limited hardware resources. Efficient architectures such as TCN have been recognized for their suitability in such low-latency environments.

Explainability and interpretability of IDS models have also gained attention in recent years. While deep learning models achieve high detection accuracy, they often operate as "black boxes," making it difficult for security analysts to understand the reasoning behind predictions. To improve trust and transparency, techniques such as SHAP (SHapley Additive exPlanations), attention visualization, and feature importance analysis are being incorporated into IDS research. These methods help identify which features contribute most to attack detection decisions.

Finally, recent literature emphasizes the importance of multi-level temporal modeling for detecting sophisticated multi-stage cyberattacks. Modern threats often evolve gradually, making them difficult to detect through static feature analysis. Hybrid models that combine convolutional, recurrent, and attention-based mechanisms have demonstrated superior performance in capturing evolving attack patterns across time. These studies collectively indicate that integrating short-term feature extraction, long-range dependency learning, and global contextual attention is crucial for building next-generation intrusion detection systems.

2.1 What is Machine Learning

Machine learning is a branch of artificial intelligence (AI) that enables computers to learn from data and make decisions without explicit programming. It identifies patterns, processes vast information, and improves accuracy over time, making it essential in fields like healthcare, finance, and automation. Unlike traditional rule-based systems, machine learning models adapt dynamically based on experience.

There are three main types of machine learning: Supervised, Unsupervised, and Reinforcement Learning. Supervised Learning uses labeled data for predictions, Unsupervised Learning finds hidden patterns in unlabeled data, and Reinforcement Learning learns by interacting with an environment through rewards and penalties. These methods enable applications like fraud detection, recommendation systems, and robotics.

Machine learning is a branch of artificial intelligence that focuses on enabling computers to learn patterns and relationships from data rather than being explicitly programmed with fixed instructions. Instead of following rigid rules, machine learning models use algorithms to analyze input data, identify trends, and make predictions or decisions. This ability allows systems to adapt to new information and improve their performance over time, making them highly useful in dynamic and data-driven environments.

At its core, machine learning works by training models on datasets that contain examples of input and corresponding outputs. Through this training process, the model learns to generalize patterns so that it can handle unseen data effectively. Depending on the type of problem, different learning approaches are used, such as supervised learning for prediction tasks, unsupervised learning for discovering hidden structures, and reinforcement learning for decision-making through interaction. These approaches enable machines to solve complex problems that are difficult to address using traditional programming methods.

Machine learning has become a key technology in modern applications across various industries. It is widely used in areas such as recommendation systems, image and speech recognition, fraud detection, healthcare diagnostics, and autonomous systems. Its ability to process large volumes of data quickly and extract meaningful insights has transformed how organizations operate and make decisions. As computational power and data availability continue to grow, machine learning is expected to play an even more significant role in shaping future technologies.

Another important aspect of machine learning is its ability to continuously improve through experience without requiring manual updates. As new data becomes available, models can be retrained or fine-tuned to adapt to changing patterns and environments. This makes machine learning highly suitable for real-time applications where conditions evolve over time, such as cybersecurity, stock market analysis, and personalized user experiences. The flexibility and scalability of machine learning systems enable them to handle complex and large-scale problems efficiently, making them a fundamental component of modern

intelligent systems.

Machine learning is revolutionizing industries by enhancing automation, decision-making, and efficiency. It powers technologies like self-driving cars, virtual assistants, and predictive analytics. As advancements in computing grow, machine learning continues to evolve, driving innovation and smarter AI solutions.

2.2 How Machine Learning Works

Machine learning works by training algorithms to recognize patterns in data and make predictions or decisions without explicit programming. The process begins with feeding raw data into a model, which extracts meaningful features and learns relationships between input and output. The model improves its accuracy over time by minimizing errors through iterative learning.

Machine learning works by allowing a computer system to learn patterns from data through a structured training process. Initially, raw data is collected and prepared by cleaning, organizing, and transforming it into a suitable format. This prepared data is then fed into a machine learning model, which uses mathematical algorithms to identify relationships between input features and expected outputs. During training, the model adjusts its internal parameters to minimize errors and improve prediction accuracy.

Once the model is trained, it is tested using new, unseen data to evaluate how well it generalizes beyond the training dataset. This phase helps determine whether the model has learned meaningful patterns or simply memorized the training data. Techniques such as validation, tuning of hyperparameters, and performance metrics like accuracy, precision, and recall are used to refine the model and enhance its effectiveness. The goal is to achieve a balance where the model performs well on both training and real-world data.

During the training phase, the machine learning algorithm analyzes the input data and identifies relationships between features and target outputs. It does this by adjusting internal parameters through mathematical optimization techniques. The goal is to minimize the difference between predicted outputs and actual results, which is achieved using loss functions and optimization methods such as gradient descent. Through multiple iterations, the model gradually improves its accuracy.

After successful training and evaluation, the model is deployed for practical use, where it can make predictions or decisions in real-time. As new data becomes available, the model can be updated or retrained to maintain its accuracy and adapt to changing conditions. This continuous learning cycle enables machine learning systems to remain relevant and effective, making them suitable for applications that require dynamic decision-making and ongoing improvement.

In FIG 2.2 shows the machine learning workflow, the input data is first preprocessed to remove noise and inconsistencies. The data is then split into training and testing sets, where the model is trained on labeled examples in supervised learning or discovers patterns in unsupervised learning. The model adjusts its internal parameters based on mathematical optimizations, such as gradient descent, to minimize prediction

errors.

Once trained, the model evaluates new, unseen data and makes predictions. Performance is assessed using metrics like accuracy, precision, and recall. If necessary, the model undergoes further fine-tuning to improve its generalization ability. Machine learning is widely applied in fields like image recognition, natural language processing, and medical diagnostics, enabling automation.

Fig.2.2: Machine Learning Workflow

In addition to training and deployment, machine learning also involves feature extraction and optimization to improve performance. Features are the important attributes or variables that help the model make accurate predictions, and selecting the right features plays a crucial role in the learning process. Advanced techniques such as dimensionality reduction and feature selection are used to remove irrelevant or redundant data, making the model more efficient and faster. By continuously refining features and optimizing algorithms, machine learning systems can achieve better accuracy, reduced complexity, and improved overall performance.

2.3 Machine Learning Paradigms

- **Supervised Learning:** Our chatbot is trained on labeled datasets containing symptoms and their corresponding diseases. Algorithms like K-Nearest Neighbors (KNN), Decision Trees, and Support Vector Machines (SVM) are used to classify user-reported symptoms and predict potential illnesses. The model learns from past cases to improve its accuracy in diagnosing medical conditions and recommending appropriate actions.

- **Unsupervised Learning:** In cases where symptoms do not match predefined disease labels, clustering

algorithms like K-Means help group similar symptom patterns. This allows the chatbot to suggest possible health conditions based on previously unseen data. Additionally, Auto encoders can be used for anomaly detection, identifying rare diseases or conditions that deviate from common patterns.

- **Reinforcement Learning:** To enhance user interaction, the chatbot can employ reinforcement learning techniques where the system continuously improves its responses based on feedback. For instance, a Deep Q-Network (DQN) can optimize chatbot replies, ensuring more accurate recommendations over time. This approach is useful in dynamically adjusting medical advice based on real-world interactions with users.

• Types of Models

- **Random Forest:** Random Forest is an ensemble machine learning algorithm that constructs multiple decision trees during training and outputs the majority class for classification. It is robust to overfitting and performs well on structured datasets. In Fig 2.3, Random Forest serves as a traditional baseline model to compare performance with deep learning architectures. However, it treats each record independently and does not capture temporal dependencies in network traffic.

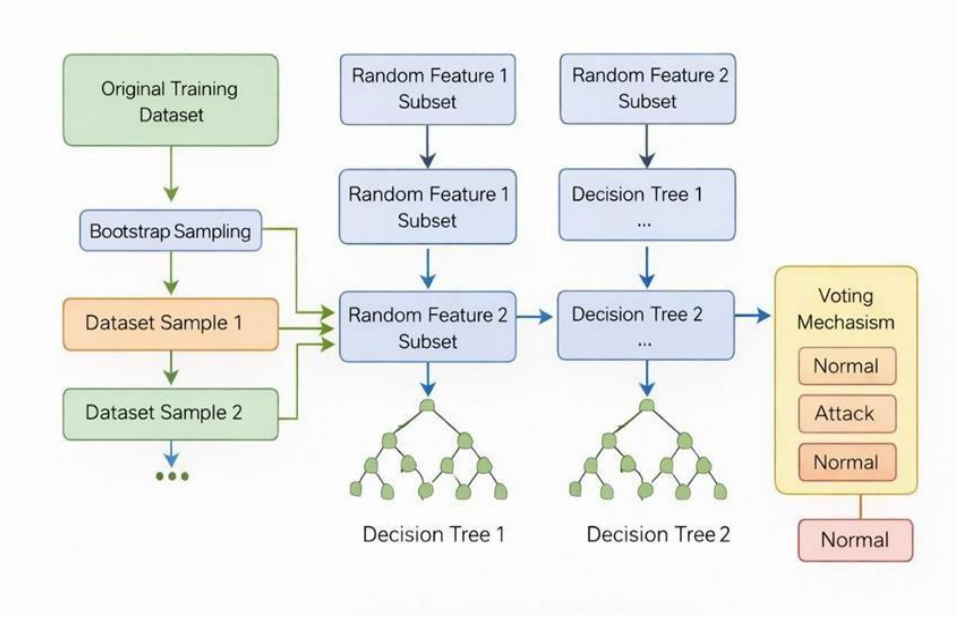

Fig.2.3*:* Working Process of Random Forest

- **CNN-LSTM:** The CNN–LSTM model is a hybrid deep learning architecture that combines the strengths of Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks. In Fig 2.4 model is particularly effective for intrusion detection because network traffic data contains both spatial relationships among features and temporal dependencies across time sequences. CNN extracts important feature patterns, while LSTM captures sequential behavior in traffic flows. The CNN–LSTM model achieved an accuracy of **96.4%**, indicating that the model correctly classified 96.4% of the total network traffic instances.

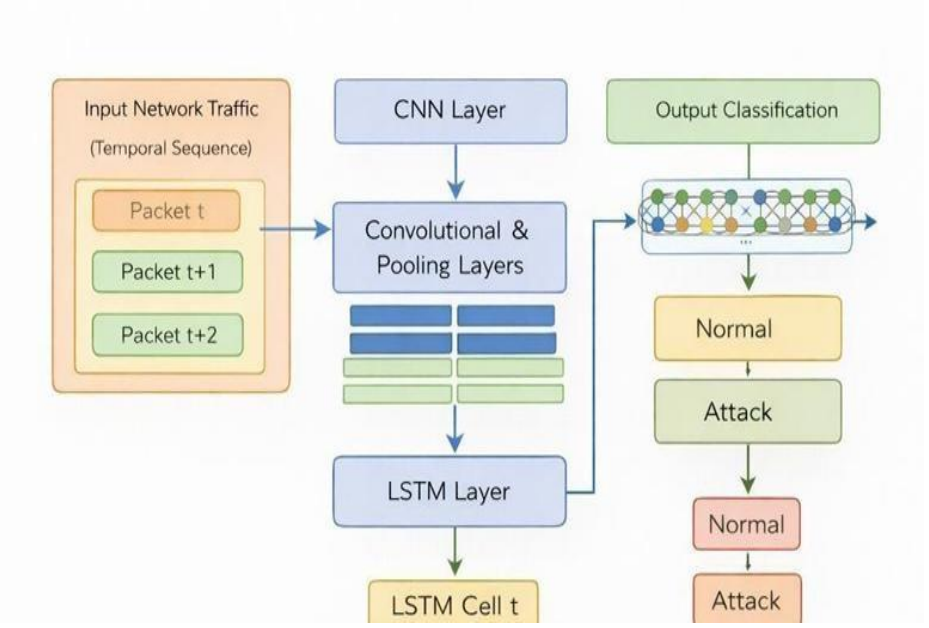

Fig 2.4: CNN-LSTM working process

• **CNN-BiLSTM**: The The CNN–BiLSTM model is a hybrid deep learning architecture that combines Convolutional Neural Networks (CNN) and Bidirectional Long Short- Term Memory (BiLSTM) networks to improve intrusion detection performance. In Fig 2.5 model is specifically designed to capture both spatial feature relationships and temporal dependencies in network traffic data. Since cyberattacks often occur as sequential and evolving patterns rather than isolated events, combining CNN and BiLSTM helps in learning complex attack behaviors more effectively.

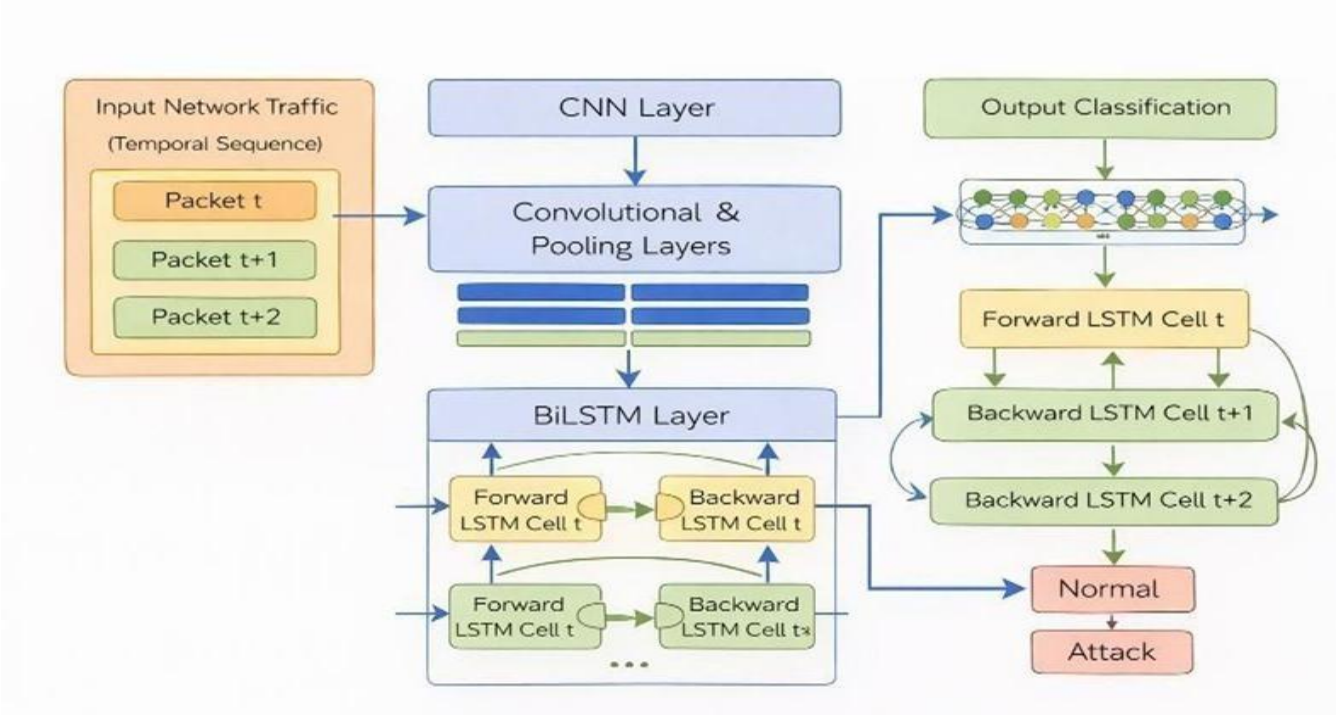

Fig 2.5: CNN-BiLSTM working process

• **TCN–BiLSTM–Transformer Model:** The proposed TCN–BiLSTM–Transformer (TBT) model is a hybrid deep learning architecture designed to enhance network intrusion detection performance by capturing short-term, long-term, and global dependencies in traffic data. Unlike traditional single-model approaches, this architecture integrates three powerful sequence-learning mechanisms to provide multi-level feature extraction and improved classification accuracy The combined TCN–BiLSTM–Transformer model achieved the highest accuracy of 98.6%, demonstrating the effectiveness of the proposed hybrid architecture. In Fig 2.6 proposed intrusion detection system introduces a hybrid deep learning framework that integrates Temporal Convolutional Networks (TCN), Bidirectional Long Short-Term Memory (BiLSTM), and Transformer Encoder to enhance detection accuracy and reliability. The key strength of the model lies in its ability to capture multi-level temporal dependencies in network traffic data, which traditional machine learning and standalone deep learning models fail to achieve effectively.

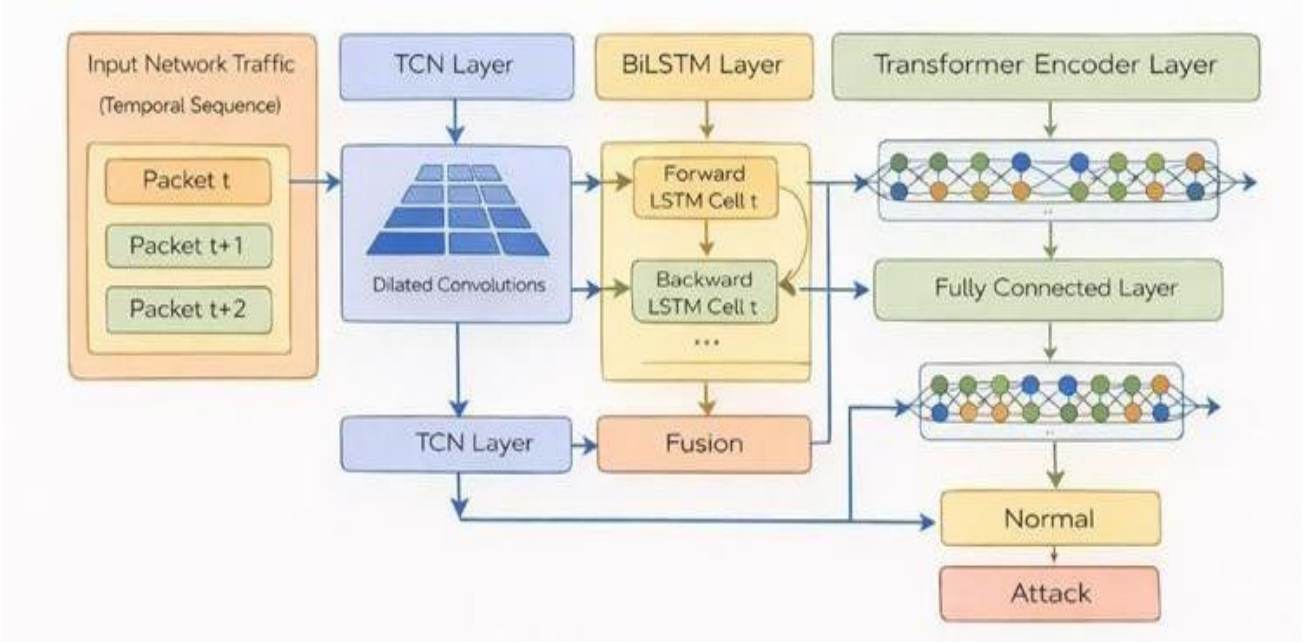

Fig 2.6: Working Process of TCN-BiLSTM-Transformer Model

2.4 Applications of Machine Learning

• **Enterprise Network Security**

Large organizations and corporate enterprises handle massive volumes of sensitive data daily. The proposed system can monitor internal and external network traffic to detect unauthorized access, malware activities, and insider threats. It helps prevent data breaches and ensures business continuity.

• **Cloud Computing Environments**

Cloud platforms host critical applications and user data. The intrusion detection model can be integrated into cloud infrastructure to monitor virtual machines, APIs, and cloud traffic for suspicious activities. This

enhances security in multi-tenant cloud environments.

- **Banking and Financial Institutions**

Banks and financial systems require strong protection against cyber threats such as phishing, DDoS attacks, and transaction fraud. The proposed IDS can detect abnormal transaction patterns and network intrusions, reducing financial risks and protecting customer information.

- **Healthcare Systems**

Hospitals and healthcare networks store confidential patient records. The intrusion detection system can protect medical databases and IoT-enabled medical devices from ransomware and unauthorized access attempts.

- **Government and Defense Networks**

Government agencies and defense organizations face highly sophisticated cyber threats. The proposed hybrid model can identify advanced persistent threats (APTs), espionage activities, and multi-stage cyberattacks in secure communication networks.

- **Internet of Things (IoT) Networks**

IoT devices generate continuous network traffic and are often vulnerable to attacks. The system can be deployed at gateways or edge devices to detect malicious behavior in smart homes, smart cities, and industrial IoT environments.

- **Data Centers and Server Farms**

Large data centers require continuous monitoring of high-speed traffic. The proposed model can analyze traffic flows in real time and prevent service disruptions caused by DDoS attacks or unauthorized intrusions.

- **E-commerce Platforms**

Online shopping platforms process large volumes of customer transactions. The intrusion detection system can detect abnormal traffic spikes, bot attacks, and fraudulent activities, ensuring secure transactions.

2.5 Characteristics of Machine Learning

- **Data Dependency:** Machine learning requires large amounts of high-quality data to improve accuracy in disease prediction and chatbot interactions.
- **Computational Complexity:** Training machine learning models can be computationally expensive, requiring powerful hardware for efficient processing.
- **Time-Intensive Training:** Depending on dataset size and complexity, training models can take significant time to achieve optimal accuracy.
- **Interpretability Challenges:** Some machine learning models, especially deep learning ones, act as black-box systems, making it difficult to understand decision-making processes.

- **Overfitting Risks:** If a model is over-trained on a specific dataset, it may perform poorly when encountering new patient data, reducing reliability.

2.6 Advantages of Machine Learning

- **High Accuracy:** Machine learning models improve healthcare chatbot accuracy by learning from vast datasets and making precise disease predictions.
- **Automated Feature Extraction:** Unlike traditional rule-based systems, machine learning automatically identifies important patterns from data.
- **Scalability:** Machine learning algorithms can handle large datasets, enabling chatbots to learn from millions of medical cases.
- **Flexibility:** The chatbot can adapt to various medical domains, supporting multiple diseases, symptoms, and user inputs.
- **Continuous Improvement:** Machine learning models continuously improve as they receive new data, making predictions more accurate over time.
- **Faster Diagnosis and Predictions:** The chatbot can instantly analyze symptoms and suggest diseases, reducing the need for lengthy consultations.
- **Handling Unstructured Data:** Machine learning enables the chatbot to process diverse input formats, including text, images, and sensor data, for better diagnosis.

2.7 Machine Learning Algorithms

Machine learning is a subset of artificial intelligence that enables systems to learn from data and make predictions.

- **Random Forest (RF)**

Random Forest is an ensemble classification algorithm used in this project as a baseline model for intrusion detection. It works by constructing multiple decision trees using random subsets of data and features, and then combining their predictions through majority voting to classify network traffic as normal or attack.

Random Forest is a powerful and widely used machine learning algorithm that belongs to the category of ensemble learning methods. It is primarily used for classification and regression tasks and is known for its high accuracy, robustness, and ability to handle large datasets. The main idea behind Random Forest is to combine multiple decision trees to produce a more reliable and stable prediction compared to a single model.

The algorithm works by creating a collection of decision trees during the training phase. Instead of using the entire dataset for each tree, Random Forest applies a technique called bootstrap sampling, where different subsets of data are selected randomly with replacement. This ensures that each

decision tree is trained on a slightly different dataset, introducing diversity among the trees and reducing the chances of overfitting.

Once all the decision trees are trained, the model makes predictions by combining the outputs of individual trees. In classification problems, the final output is determined by majority voting, where the class predicted by most trees is selected as the final result. In regression problems, the predictions of all trees are averaged to produce the final output. This aggregation process enhances accuracy and reduces variance.

Components of Random Forest:

- **Bootstrap Sampling:** Creates multiple subsets of training data using sampling with replacement.
- **Random Feature Selection:** Selects a random subset of features at each split to reduce correlation among trees.
- **Decision Trees:** Builds multiple independent trees for classification.
- **Majority Voting:** Final prediction is determined by the class predicted by most trees.

Applications:

- Network intrusion detection.
- Baseline comparison for deep learning models.
- Feature importance analysis.

- **Convolutional Neural Network (CNN)**

CNN is a deep learning model used for extracting spatial feature patterns from structured network traffic data. It identifies correlations between traffic attributes and detects local feature interactions. A Convolutional Neural Network (CNN) is a type of deep learning algorithm specifically designed to process structured grid-like data such as images, time-series data, and network traffic features. It is widely used in applications like image recognition, video analysis, natural language processing, and intrusion detection systems. CNNs are highly effective in automatically extracting important features from input data without the need for manual feature engineering.

The architecture of a CNN consists of multiple layers, each performing a specific function to transform the input data into meaningful representations. The primary component is the convolutional layer, where filters (also known as kernels) are applied to the input data to detect patterns such as edges, textures, or feature relationships. These filters slide across the input, performing mathematical operations that capture local patterns and spatial dependencies.

After the convolutional layer, an activation function such as ReLU (Rectified Linear Unit) is applied to introduce non-linearity into the model. This allows the network to learn complex

patterns beyond simple linear relationships. The output of the activation function is then passed to a pooling layer, which reduces the dimensionality of the data while preserving important features. Pooling helps in decreasing computational complexity and prevents overfitting by summarizing feature information.

As the data moves deeper into the network, multiple convolutional and pooling layers work together to extract increasingly complex features. Initial layers may capture simple patterns, while deeper layers identify more abstract and high-level features. This hierarchical feature extraction makes CNNs highly efficient for pattern recognition tasks.

At the end of the network, fully connected layers are used to perform classification or prediction. These layers take the extracted features and map them to output classes. A final activation function such as softmax or sigmoid is applied to generate probabilities for each class, enabling the model to make decisions.

Components of CNN:

- **Convolution Layer:** Applies filters to extract important local patterns.
- **Activation Function (ReLU):** Introduces non-linearity.
- **Pooling Layer:** Reduces dimensionality while preserving key features.
- **Fully Connected Layer:** Performs classification.

- **CNN–LSTM**

CNN–LSTM is a hybrid deep learning architecture combining CNN for spatial feature extraction and LSTM for sequential learning of network traffic behavior over time.

Components of CNN-LSTM:

- **CNN Layer:** Extracts spatial features.
- **LSTM Layer:** Captures temporal dependencies in sequential traffic data.
- **Dense Layer:** Performs final classification.
- **Softmax/Sigmoid Output:** Produces classification probabilities.

Applications:

- Detection of time-based cyberattacks.
- Sequential traffic behavior analysis.

- **CNN–BiLSTM**

CNN–BiLSTM extends CNN–LSTM by replacing LSTM with Bidirectional LSTM to improve contextual understanding by learning both forward and backward dependencies. The CNN–BiLSTM model is a hybrid deep learning architecture that combines the strengths of Convolutional

Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) networks. This model is specifically designed to handle data that contains both spatial and temporal characteristics. By integrating these two powerful techniques, the model can effectively analyze complex patterns in sequential data such as network traffic, making it highly suitable for intrusion detection systems.

The CNN component is responsible for extracting spatial features from the input data. It applies convolutional filters to identify important local patterns and relationships among features. In the context of network traffic, CNN can capture correlations between attributes such as packet size, protocol type, and flow duration. These features are essential for identifying suspicious behavior at a local level. The pooling layers further refine the extracted features by reducing dimensionality and removing redundant information.

Once the spatial features are extracted, they are passed to the BiLSTM component for sequential analysis. Unlike traditional LSTM, which processes data in a single direction, BiLSTM analyzes the sequence in both forward and backward directions. This allows the model to understand how past and future data points influence the current state. In intrusion detection, this capability is crucial because cyberattacks often occur as sequences of events rather than isolated incidents.

The combination of CNN and BiLSTM enables the model to capture both short-term and long-term dependencies in data. CNN focuses on local feature extraction, while BiLSTM captures temporal relationships across sequences. This dual learning mechanism enhances the model's ability to detect complex and multi-stage attack patterns that may not be identified by standalone models.

The architecture typically includes convolutional layers followed by pooling layers, which feed into one or more BiLSTM layers. The outputs from the BiLSTM are then passed through fully connected layers for final classification. Activation functions such as softmax or sigmoid are used to generate prediction probabilities, allowing the system to classify input data as normal or malicious.

Key Features of CNN-BiLSTM:

- **CNN Layer:** Extracts local spatial patterns.
- **Forward LSTM:** Processes sequence from start to end.
- **Backward LSTM:** Processes sequence from end to start.
- **Feature Concatenation:** Combines forward and backward outputs.
- **Dense Layer:** Performs classification.

Applications:

- Multi-stage attack detection.
- Context-aware intrusion detection systems.

- **Bidirectional Long Short-Term Memory (BiLSTM)**

BiLSTM is a recurrent neural network that processes sequential data in both forward and backward directions to capture long-range dependencies.

Key Features of BiLSTM:

- **Conditional Probability Estimation:** Calculates disease probability based on past cases.
- **Feature Independence Assumption:** Assumes symptoms occur independently (though not always true in medical cases).
- **Forward LSTM Cell:** Learns past-to-future dependencies.
- **Backward LSTM Cell:** Learns future-to-past dependencies.
- **Memory Gates:** Input gate, forget gate, and output gate.
- **Concatenation Layer:** Combines outputs from both directions.

Applications:

- Sequential pattern recognition.
- Detection of evolving cyberattacks.

- **TCN–BiLSTM–Transformer**

This is the final proposed model that integrates TCN, BiLSTM, and Transformer to capture short-term, long-term, and global dependencies simultaneously. The TCN–BiLSTM–Transformer model is an advanced hybrid deep learning architecture designed to improve the performance of complex data analysis tasks such as network intrusion detection. This model combines three powerful components—Temporal Convolutional Networks (TCN), Bidirectional Long Short-Term Memory (BiLSTM), and Transformer encoders—to capture multi-level patterns in data. By integrating these techniques, the model is capable of analyzing short-term variations, long-term dependencies, and global contextual relationships within sequences.

The first component, the Temporal Convolutional Network (TCN), is responsible for capturing short-term temporal features. TCN uses dilated causal convolutions, which allow the model to process sequences efficiently while maintaining the order of data. Unlike traditional recurrent networks, TCN supports parallel computation and avoids issues such as vanishing gradients. It effectively identifies local patterns and immediate changes in the input data, making it suitable for detecting sudden anomalies in network traffic.

The second component, the Bidirectional Long Short-Term Memory (BiLSTM), focuses on learning long-term dependencies in sequential data. BiLSTM processes the input sequence in both

forward and backward directions, allowing the model to understand the context from past and future time steps simultaneously. This bidirectional learning enhances the model's ability to detect patterns that evolve over time, such as multi-stage cyberattacks or gradual changes in network behavior.

The final component, the Transformer encoder, introduces an attention mechanism that enables the model to focus on the most important parts of the sequence. Unlike recurrent models, the Transformer does not rely on sequential processing; instead, it evaluates relationships between all elements in the sequence simultaneously. This allows it to capture global dependencies and interactions that may be missed by other models. The attention mechanism assigns different weights to different features, ensuring that critical information is emphasized during prediction.

The integration of these three components creates a powerful multi-layer architecture. The TCN extracts local temporal features, the BiLSTM captures sequential dependencies, and the Transformer refines the representation by modeling global relationships. The combined output is then passed through fully connected layers for classification, enabling accurate prediction of outcomes such as normal or malicious network activity.

Components of TCN–BiLSTM–Transformer

- **TCN Module:** Extracts short-term temporal features.
- **BiLSTM Module:** Learns bidirectional long-term dependencies.
- **Transformer Encoder:** Captures global contextual relationships.
- **Feature Fusion Layer:** Combines outputs from all modules.
- **Fully Connected Output Layer:** Performs final classification.

Applications:

- High-accuracy intrusion detection (98.6%).
- Enterprise and cloud network security.
- Detection of rare and multi-stage cyberattacks.

2.8 Architectural Methods for Machine Learning Algorithms

Architectural methods in machine learning refer to the structural design and organization of models used to learn patterns from data. In intrusion detection systems, selecting appropriate architectural methods is essential for effectively capturing spatial features, temporal dependencies, and global contextual relationships in network traffic data. Different machine learning and deep learning architectures are designed to address specific challenges such as high dimensionality, sequential behavior, and class imbalance.

- **Traditional Machine Learning Architectures**

Backpropagation Traditional machine learning architectures such as Decision Trees, Random

Forest, Support Vector Machines (SVM), and k-Nearest Neighbors (KNN) follow a structured feature-based approach. These models rely on manually engineered features extracted from network traffic data. The architecture typically consists of input feature vectors, a learning algorithm for pattern recognition, and an output classification layer. Although computationally efficient and interpretable, these models do not inherently capture sequential or temporal dependencies in data.

- **Convolutional Neural Network (CNN) Architecture**

CNN architecture is designed to automatically extract spatial features from structured input data. It consists of convolutional layers, activation functions (ReLU), pooling layers, and fully connected layers. In intrusion detection, CNN captures correlations among network traffic attributes and identifies local feature patterns. This architecture is effective for high- dimensional datasets but lacks strong temporal modeling capability.

- **Bidirectional LSTM (BiLSTM) Architecture**

BiLSTM extends the LSTM architecture by processing sequences in both forward and backward directions. This dual-direction learning improves contextual understanding and enhances the detection of complex, multi-stage cyberattacks. The architecture consists of forward LSTM, backward LSTM, concatenation of outputs, and a classification layer.

- **Temporal Convolutional Network (TCN) Architecture**

TCN is a specialized sequence modeling architecture that uses dilated causal convolutions and residual connections. Unlike RNNs, TCN allows parallel computation and maintains long effective memory through dilation. It captures short-term temporal dependencies efficiently and avoids vanishing gradient problems common in recurrent networks.

- **Hybrid Architectural Methods**

Hybrid architectures combine multiple learning mechanisms to overcome limitations of individual models. In this project, the TCN–BiLSTM–Transformer architecture integrates:

- TCN for short-term temporal feature extraction
- BiLSTM for long-range sequential dependency learning
- Transformer encoder for global attention-based modeling

This multi-level architectural design ensures comprehensive feature extraction, improved detection accuracy, and better generalization performance.

2.9 Libraries/Frameworks of Machine Learning

Several Machine learning frameworks are software libraries and platforms that provide tools, algorithms, and utilities to develop, train, test, and deploy machine learning models efficiently. These frameworks simplify complex mathematical computations, support large- scale data processing, and enable rapid model development.

- **Scikit-learn**
- Provides simple and efficient tools for machine learning.
- Used for data preprocessing, model selection, and evaluation.
- Supports classification, regression, and clustering models.
- Works well with other Python libraries like NumPy and Pandas.
- **TensorFlow**
- Developed by Google for deep learning and machine learning applications.
- Supports neural networks and advanced AI models.
- Efficiently processes large medical datasets.
- Provides **TensorFlow Lite** for mobile deployment.
- **Keras**
- A high-level API built on TensorFlow, simplifying model creation.
- Supports modular and user-friendly neural network design.
- Allows quick prototyping for machine learning models.
- **PyTorch**
- Developed by Facebook for dynamic computational graphs.
- Efficient for medical image processing and NLP-based chatbot applications.
- Optimized for GPU acceleration with **CUDA support**.
- **Pandas**
- Used for data manipulation and analysis.
- Helps in loading, cleaning, and structuring healthcare datasets.
- Essential for handling symptom and disease-related data.
- **NumPy**
- Provides support for large multidimensional arrays.
- Optimizes mathematical computations for machine learning models.
- Used in processing numerical healthcare data.
- **Matplotlib & Seaborn**
- Used for visualizing disease prediction trends.
- Helps in analyzing and interpreting symptom-disease correlations.

2.10 Common Steps for TCN–BiLSTM–Transformer

Step 1: Import Necessary Libraries

First, we import the essential libraries required for data processing, model building, and evaluation.

```
Import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, cross_val_score from
sklearn import preprocessing
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

Step 2: Load and Preprocess Data

We load the dataset and prepare it by encoding categorical labels and splitting it into training and testing sets.

```
# Load dataset
data = pd.read_csv("dataset.csv") # Replace with your dataset file #
Extract features (X) and target labels (y)
X = data.iloc[:, :-1] # All columns except the last one y =
data.iloc[:, -1] # Target column (last column)
# Encode target labels if they are categorical le =
preprocessing.LabelEncoder()
y = le.fit_transform(y)
# Split data into training and testing sets (70% training, 30% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 3: Train the TCN-BiLSTM-Transformer Model We train the Decision Tree classifier using the training data.

```
# Initialize and train the Decision Tree Classifier
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
```

Step 4: Evaluate the Model

We test the model using the testing dataset and calculate its accuracy.

```
# Evaluate model on test data
accuracy = clf.score(X_test, y_test)
```

print(f”Decision Tree Classifier Accuracy: {accuracy:.2f}”)

We can also perform **cross-validation** to check the model’s consistency: #

Perform 5-fold cross-validation

scores = cross_val_score(clf, X, y, cv=5)

print(f”Cross-validation Accuracy: {scores.mean():.2f}”)

Step 5: Make Predictions

We use the trained model to make predictions on new data. #

Predict on test data

y_pred = clf.predict(X_test)

Predict for a new sample (example)

sample_data = np.array([1, 0, 1, 1, 0]) # Example input (modify as needed)

sample_data = sample_data.reshape(1, -1)

prediction = clf.predict(sample_data)

print(f”Predicted class: {le.inverse_transform(prediction)[0]}”)

Step 6: Extract Feature Importance

We can check which features are most important in making predictions. # Get

feature importances

feature_importances = clf.feature_importances_

features = X.columns

Display feature importance ranking

sorted_indices = np.argsort(feature_importances)[::-1]

print(“Feature Importance Ranking:”)

for idx in sorted_indices:

print(f”{features[idx]}: {feature_importances[idx]:.4f}”)

3. SYSTEM ANALYSIS & DESIGN

3.1 Scope of the Project

The scope of this project focuses on designing and implementing an advanced Network Intrusion Detection System (IDS) using a hybrid deep learning architecture combining TCN, BiLSTM, and Transformer models. The system aims to accurately detect malicious network activities by analyzing structured network traffic datasets such as UNSW-NB15 and NSL-KDD. The project primarily addresses the challenge of identifying both common and rare cyberattacks by capturing short-term, long-term, and global dependencies in traffic data. The project scope includes data preprocessing techniques such as normalization, encoding, feature selection using BRFE, sliding-window segmentation, and handling class imbalance using SMOTE. These preprocessing steps enhance the quality of input data and improve model generalization. The implementation also involves training and evaluating multiple baseline models (Random Forest, CNN, CNN-LSTM, CNN- BiLSTM) for performance comparison.

In addition to model development and evaluation, the project scope also includes the design and implementation of a user-friendly interface for real-time interaction with the intrusion detection system. This interface enables users to upload data, initiate detection, and visualize results in an intuitive format, thereby bridging the gap between complex model outputs and practical usability. The system is structured to support both offline analysis using benchmark datasets and online inference for real-time traffic monitoring.

The project further encompasses performance optimization and validation under different experimental settings. Various hyperparameters such as learning rate, batch size, and sequence length are tuned to achieve optimal model performance. Cross-validation and repeated experiments are conducted to ensure the reliability and consistency of results. This systematic evaluation helps in identifying the most efficient configuration for accurate intrusion detection.

Another important aspect within the scope is the comparative analysis of different models to understand their strengths and limitations. By evaluating traditional machine learning and deep learning approaches alongside the proposed hybrid model, the project provides insights into how different architectures perform under the same conditions. This analysis supports the justification for selecting a hybrid approach.

The scope also includes the assessment of model scalability and computational efficiency. The system is designed to handle high-dimensional network traffic data and large datasets without significant degradation in performance. Consideration is given to memory usage, training time, and inference speed, ensuring that the proposed solution remains practical for real-world deployment scenarios.

Furthermore, the project explores the adaptability of the model to different types of network environments. By using multiple datasets and diverse traffic patterns, the system is evaluated for its ability to generalize across varying conditions. This ensures that the model is not limited to a specific dataset but can be extended to broader cybersecurity applications. Finally, the project scope lays the foundation for future enhancements by providing a modular and extensible framework. The architecture can be expanded to incorporate additional features such as real-time alert systems, integration with security tools, and support for multi-class classification. This flexibility ensures that the proposed system can evolve with emerging cybersecurity challenges and technological advancements.

In addition to training and deployment, machine learning also involves feature extraction and optimization to improve performance. Features are the important attributes or variables that help the model make accurate predictions, and selecting the right features plays a crucial role in the learning process. Advanced techniques such as dimensionality reduction and feature selection are used to remove irrelevant or redundant data, making the model more efficient and faster. By continuously refining features and optimizing algorithms, machine learning systems can achieve better accuracy, reduced complexity, and improved overall performance.

Another important aspect of the project scope is the implementation of advanced data processing techniques to improve model performance. This includes data cleaning, transformation, and selection of relevant features to ensure that the model learns from high-quality input. The project also covers the training and evaluation of multiple models to compare their effectiveness and identify the most suitable approach for intrusion detection. These steps help in achieving a balance between accuracy, efficiency, and reliability.

3.2 Analysis

Initially, the UNSW-NB15 dataset was analyzed to understand its structure, feature distribution, and class imbalance. The dataset contains 175,341 records with 36 features representing network traffic attributes such as packet size, protocol type, service, and flow duration. Label distribution analysis revealed an imbalance between normal and attack classes, highlighting the need for oversampling techniques such as SMOTE to improve minority class detection.

Feature analysis was performed using Boosted Recursive Feature Elimination (BRFE) to identify the most important attributes contributing to attack detection. Redundant and less informative features were removed to reduce dimensionality and improve computational efficiency. This optimization step enhanced model training speed and reduced overfitting. The dataset used for training includes structured information on disease symptoms, their severity levels, and associated precautions**.** This allows the chatbot to analyze symptom patterns and provide an initial diagnosis, making it a valuable tool for preliminary healthcare

The Fig 3.1 illustrated intrusion detection model presents a structured pipeline that separates feature learning into specialized components before making a final decision. This modular design enhances the system's ability to capture different aspects of network traffic behavior in a coordinated manner.

The process begins with feature extraction, where raw network traffic is transformed into meaningful representations suitable for model input. These extracted features are then processed through two parallel analytical paths. The first path, the *Local Pattern Analyzer*, focuses on identifying fine-grained patterns and short-term variations within the data. This component is responsible for capturing localized anomalies such as sudden spikes in packet activity or irregular protocol usage, which are often indicative of immediate or surface- level attack signatures.

The second path, the *Temporal Behavior Module*, is designed to analyze the sequential and time-dependent nature of network traffic. Instead of focusing on isolated instances, this module examines how traffic evolves over time, enabling the detection of coordinated or multi-stage attacks. Such attacks may not appear suspicious at a single point but reveal malicious intent when viewed as a sequence of actions.

Following these parallel analyses, the outputs are combined in the *Feature Fusion* stage. This step integrates both local and temporal insights into a unified representation. By merging these complementary features, the system gains a more holistic understanding of network behavior. The fusion process ensures that neither short-term anomalies nor long- term dependencies are overlooked, thereby improving the robustness of the detection mechanism.

The final stage, the *Decision Network*, performs classification based on the fused features. This component applies learned decision boundaries to determine whether the observed traffic is normal or malicious. The use of a dedicated decision layer allows the system to effectively interpret the combined feature space and produce accurate predictions.

An important strength of this architecture is its parallel processing capability. By analyzing local and temporal characteristics simultaneously, the system reduces the risk of missing critical attack indicators. Additionally, the modular structure allows for flexibility, making it easier to upgrade individual components without affecting the entire system. Overall, the model demonstrates a well-balanced approach to intrusion detection by combining multiple perspectives of data analysis.

The analysis phase of this project focuses on understanding the structure and characteristics of network traffic data to build an effective intrusion detection system. It involves examining the dataset to identify important attributes, patterns, and relationships that can help distinguish between normal and malicious activities. By carefully studying the data, meaningful insights are obtained that guide the design of the model and improve its decision-making capability.

Another key part of the analysis is evaluating the distribution and quality of the data. This includes identifying issues such as missing values, noise, and class imbalance, which can negatively impact model

performance. Techniques are applied to handle these challenges and ensure that the dataset is well-prepared for training. Additionally, statistical and visual analysis methods are used to better understand feature behavior and their influence on the final predictions. The analysis also involves examining how different components of the system interact during the detection process. It helps in defining the workflow, from data input to final classification, and ensures that each stage contributes effectively to the overall system performance. By conducting a thorough analysis, the project establishes a strong foundation for building a robust and accurate intrusion detection model.

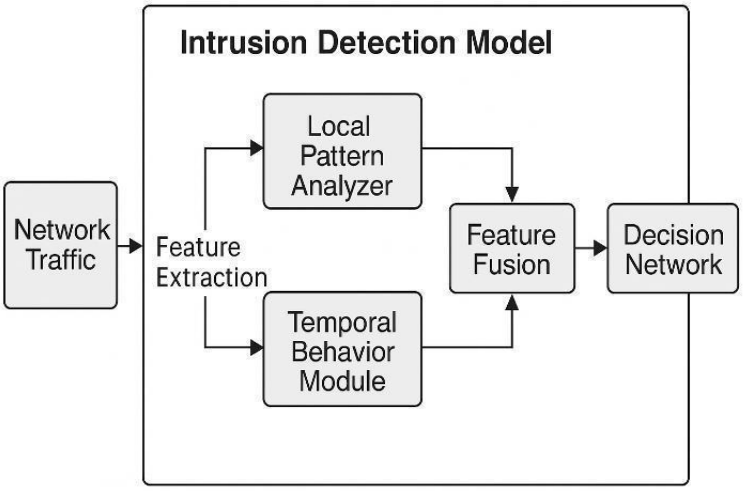

Fig 3.1 Architecture diagram

3.3 Dataset Description

The Experiments are performed using publicly available benchmark network intrusion datasets. These datasets have records of network traffic, reflecting various kinds of normal and malicious activities. Each record comprises various attributes in terms of numbers, reflecting behaviors and features of the packets, and network and protocol details. The datasets are loaded from CSV files and divided into training and testing sets using an 80:20 split to ensure proper model evaluation. This is done to make sure that both training and test sets are properly distinguished before proceeding with any operations. The dataset used in this project is the **UNSW-NB15** dataset, which is a modern benchmark dataset designed for network intrusion detection research. It was created by the Australian Centre for Cyber Security (ACCS) to overcome the limitations of older datasets such as KDD'99 and NSL- KDD. The UNSW-NB15 dataset contains realistic synthetic network traffic that includes both normal activities and various types of cyberattacks. The dataset consists of **175,341 records** with **36 features**, where each record represents a network flow. These features include both numerical and categorical attributes such as source and destination IP addresses, protocol type, service type, packet size, duration, number of bytes transferred, and flow behavior statistics. The dataset also includes a binary label indicating whether the traffic

3.4 Data Preprocessing

Data preprocessing is a crucial step in building an effective intrusion detection system. Since raw network traffic data often contains missing values, mixed data types, redundant features, and class imbalance, proper preprocessing is required to improve model accuracy and reliability. In this project, several preprocessing techniques were applied to prepare the UNSW-NB15 dataset for training the TCN–BiLSTM–Transformer model.

The first step in preprocessing involves handling missing, inconsistent, or duplicate records. Any null or undefined values present in the dataset were either removed or appropriately handled to prevent bias during model training. Cleaning ensures that the dataset is consistent and reliable for further analysis.

The dataset contains categorical attributes such as protocol type, service, and state. Since machine learning models require numerical inputs, these categorical variables were converted into numerical form using one-hot encoding. This technique creates binary columns for each category, allowing the model to process categorical information effectively. Network traffic features such as packet size, duration, and byte count vary widely in scale. To ensure uniform contribution of all features, Min–Max normalization was applied. This technique scales numerical values to a fixed range (typically 0 to 1), preventing features with larger values from dominating the learning process. High-dimensional data may contain redundant or less important features that increase computational complexity. To address this, Boosted Recursive Feature Elimination (BRFE) was used. This method ranks features based on importance scores and iteratively removes less significant attributes. Feature selection reduces overfitting, improves training efficiency, and enhances generalization.

The UNSW-NB15 dataset exhibits class imbalance between normal and attack records. To ensure balanced learning, the Synthetic Minority Oversampling Technique (SMOTE) was applied. SMOTE generates synthetic samples for minority classes, improving recall for rare attack categories and reducing bias toward majority classes.

Since cyberattacks often occur as sequential events, sliding-window segmentation was applied to convert traffic records into temporal sequences. This enables the model to capture short-term and long-term dependencies using TCN and BiLSTM layers.

Effective preprocessing improves model stability, reduces noise, enhances feature quality, and significantly boosts detection accuracy. Without proper preprocessing, deep learning models may overfit, underperform, or misclassify minority attack types.

Fig 3.2: Preprocessing Techniques on Feature Distributions

The Fig 3.2 illustrates the outcome of feature analysis performed using multiple machine learning models, including Linear Regression, Decision Tree, Random Forest, and XGBoost. This stage plays a crucial role in data preprocessing by identifying the relative importance and contribution of different input features before feeding them into the final intrusion detection model.

Each model evaluates feature significance differently, providing diverse perspectives on how input variables influence prediction outcomes. In Linear Regression, feature coefficients indicate both the direction and strength of influence. Positive coefficients show features that contribute directly to prediction, while negative values highlight inverse relationships. This helps in understanding which attributes increase or decrease the likelihood of a particular classification.

The Decision Tree model ranks features based on their ability to split the data effectively. Features with higher importance scores are those that contribute most to reducing uncertainty during classification. This hierarchical evaluation helps identify dominant attributes that play a key role in decision-making.

Random Forest, being an ensemble of multiple decision trees, provides a more stable and generalized estimation of feature importance. It reduces the bias of individual trees and highlights features that consistently contribute to accurate predictions across different subsets of data. This makes it particularly useful for identifying robust and reliable features.

XGBoost further refines this process by using gradient boosting techniques, where features are evaluated based on their contribution to minimizing prediction error during iterative training. It captures complex interactions between variables and often identifies subtle but impactful features that may not be evident

3.5 Model Comparison

Model comparison is an essential step in evaluating the effectiveness of different machine learning and deep learning algorithms for intrusion detection. In this project, five models were compared: Random Forest, CNN, CNN–LSTM, CNN–BiLSTM, and the proposed TCN–BiLSTM–Transformer model. The comparison was performed using key evaluation metrics such as Accuracy, Precision, and F1-Score.

The Random Forest model was used as a traditional baseline approach. Although it provides stable and interpretable results, it achieved lower performance compared to deep learning models. Since Random Forest does not capture temporal dependencies in network traffic data, its accuracy was limited to 87.2%, making it less suitable for detecting multi- stage or time-based attacks.

Model comparison is an essential process in machine learning that helps determine which algorithm performs best for a given problem. Instead of relying on a single model, multiple models are trained and evaluated under the same conditions to ensure a fair assessment. This approach provides a clear understanding of how different algorithms behave when exposed to the same dataset and helps in selecting the most suitable model for accurate predictions.

One important aspect of model comparison is the use of evaluation metrics to measure performance. Metrics such as accuracy, precision, recall, and F1-score are commonly used to assess how well a model classifies data. Each metric highlights a different aspect of performance, allowing a more detailed analysis. For example, some models may achieve high accuracy but perform poorly in identifying minority classes, which becomes evident when considering precision and recall.

The CNN model showed significant improvement over Random Forest by effectively extracting spatial features from structured network traffic data. It achieved 95.8% accuracy and high precision (99.7%), indicating strong ability to detect attack patterns. However, CNN alone cannot model long-term sequential behavior in traffic flows.

The CNN–LSTM and CNN–BiLSTM models further improved performance by incorporating sequential learning. The CNN–LSTM model achieved 96.4% accuracy, while CNN–BiLSTM slightly outperformed it with 96.8% accuracy due to bidirectional learning capability. These models effectively captured time-dependent attack patterns but still had limitations in modeling global contextual relationships.

The proposed TCN–BiLSTM–Transformer model achieved the highest performance among all models. With 98.6% accuracy, 99.7% precision, and 99.07% F1-score, it demonstrated superior detection capability. The improvement is mainly due to multi-level temporal learning: TCN captures short-term dependencies, BiLSTM learns long-term sequential relationships, and Transformer models global contextual interactions using self- attention. Overall, the comparison clearly shows that hybrid deep learning architectures outperform traditional machine learning methods in intrusion detection tasks. The proposed model provides higher accuracy, better minority attack detection, and lower false alarm rates,

making it more suitable for real-world cybersecurity applications.

Another factor considered during comparison is the learning capability of the model. Some models are better at capturing simple patterns, while others can learn complex relationships within the data. Advanced models, especially deep learning architectures, tend to perform better in handling large and complex datasets. However, simpler models may still be useful due to their interpretability and lower computational requirements.

Computational efficiency and scalability also play a significant role in model comparison. Some algorithms require more training time and resources, making them less suitable for real-time applications. Evaluating how quickly a model trains and makes predictions is important, especially in systems where speed and responsiveness are critical. A balance between performance and efficiency is necessary for practical deployment.

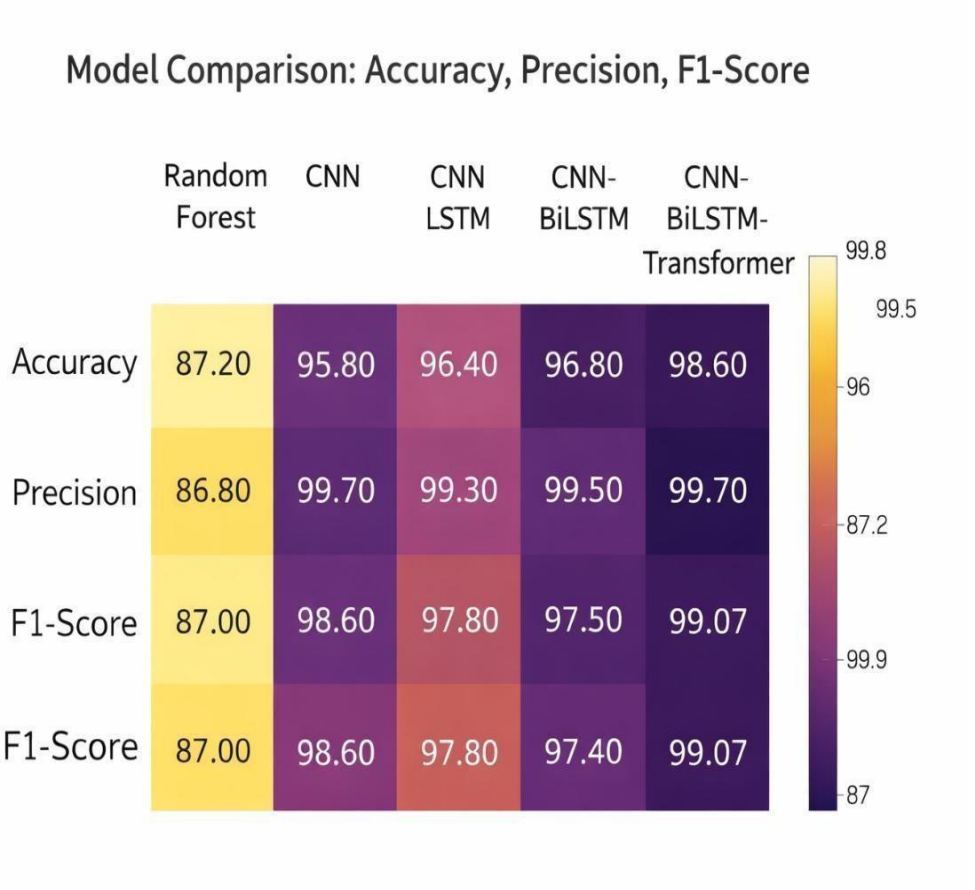

Fig 3.3: Comparison Between Models

Fig 3.3 presented heatmap offers a comprehensive visualization of performance variations across multiple models using key evaluation metrics. By representing accuracy, precision, and F1-score in a color-coded format, the figure enables an intuitive understanding of how each model performs relative to others. Darker shades indicate higher performance, allowing quick identification of the most effective model.

One of the key insights from the figure is the progressive improvement in performance as models evolve

from traditional machine learning approaches to more advanced hybrid deep learning architectures. This progression reflects the increasing ability of models to capture complex patterns in network traffic data. As cyberattacks become more sophisticated, the need for models capable of understanding intricate relationships and dependencies becomes critical.

The variation in performance across models also highlights the importance of feature representation and learning mechanisms. Simpler models rely heavily on predefined patterns and statistical relationships, whereas deep learning models automatically learn hierarchical features. This shift from manual to automated feature extraction significantly enhances detection capabilities, particularly for complex and evolving attack patterns.

Another important aspect revealed by the figure is the consistency of performance across different evaluation metrics. Models that maintain uniformly high values across accuracy, precision, and F1-score are generally more reliable, as they demonstrate balanced performance in detecting both positive and negative classes. In contrast, models with uneven metric distributions may perform well in one aspect but struggle in others, indicating potential weaknesses in real-world scenarios.

The figure also emphasizes the role of sequential learning in intrusion detection. Models that incorporate temporal components show noticeable improvements, suggesting that analyzing the order and timing of events is crucial for identifying malicious activities. Network intrusions often occur as sequences of actions rather than isolated events, making temporal modeling an essential requirement.

Furthermore, the visualization highlights how hybrid models achieve superior performance by integrating multiple learning strategies. Instead of relying on a single mechanism, hybrid architectures combine different perspectives of data analysis, resulting in a more comprehensive understanding of network behavior. This integration allows the model to detect subtle and complex attack patterns that may not be captured by individual models. From an analytical standpoint, the small performance differences between advanced models indicate that the system is approaching an optimal learning boundary. Achieving further improvements at this stage requires more sophisticated techniques, such as attention mechanisms, advanced feature engineering, or enhanced training strategies. The proposed model's ability to outperform others in this competitive range demonstrates its effectiveness and robustness.

Another key observation is the reduced performance variability in the proposed model. The tight clustering of metric values indicates that the model performs consistently across different evaluation criteria. This stability is particularly important in cybersecurity applications, where inconsistent performance can lead to missed detections or excessive false alarms. The figure also provides insights into error distribution and model confidence indirectly. High precision values suggest that the models are effective in minimizing false positives, while strong F1-scores indicate a good balance between precision and recall. This balance is critical in intrusion detection, where both false alarms and missed attacks can have serious

consequences.

Another important dimension in model comparison is the ability of models to generalize well to unseen data. A model that performs exceptionally on training data but poorly on new data is not reliable for real-world applications. Therefore, techniques such as cross-validation are used to evaluate how consistently a model performs across different subsets of data. Models with better generalization capability are preferred, as they can handle variations in input data more effectively.

Model interpretability is also a key consideration during comparison. Some models, such as decision trees and linear models, provide clear insights into how decisions are made, making them easier to understand and trust. In contrast, more complex models like deep neural networks often act as black boxes, where the decision-making process is not easily explainable. Depending on the application, especially in critical domains, choosing a model that balances performance with interpretability becomes important.

In addition, the comparison highlights the importance of model architecture design in achieving high performance. The transition from single-layer models to multi-component hybrid systems demonstrates how combining different computational approaches leads to better learning outcomes. This architectural evolution reflects current trends in deep learning, where hybrid and ensemble methods are increasingly used to address complex problems. From a practical perspective, the results shown in the figure suggest that deploying advanced hybrid models can significantly enhance the effectiveness of intrusion detection systems. The improved performance not only increases detection accuracy but also enhances system reliability and trustworthiness. This is particularly important in real- world environments where security decisions must be made quickly and accurately.

Additionally, robustness to noise and data variations is another factor that influences model selection. Real-world data is rarely perfect and often contains errors or unexpected patterns. A good model should be able to maintain stable performance even when the input data is slightly altered or contains noise. Comparing models under such conditions helps identify those that are more resilient and reliable, ensuring consistent performance in practical environments.

Finally, model comparison helps in understanding the strengths and limitations of each approach. By analyzing different models, it becomes easier to identify which one is more robust, reliable, and adaptable to changing data conditions. This process not only improves the overall system design but also provides insights for future improvements, ensuring that the selected model delivers consistent and high-quality results.

```
import matplotlib.pyplot as plt import numpy as np
import pandas as pd
# Model names from your project
models = ['Random Forest', 'CNN', 'CNN-LSTM', 'CNN-BiLSTM', 'TCN-BiLSTM-Transformer']
# Metric values from your project results (Accuracy, Precision, F1-Score) data = {
```

```
'Random Forest': [87.2, 86.8, 87.0],
'CNN': [95.8, 99.7, 98.6],
'CNN-LSTM': [96.4, 99.3, 97.8],
'CNN-BiLSTM': [96.8, 99.5, 97.4],
'TCN-BiLSTM-Transformer': [98.6, 99.7, 99.07]
}

# Create DataFrame
df = pd.DataFrame(data, index=['Accuracy', 'Precision', 'F1-Score'])

# Create figure plt.figure(figsize=(10, 6))

# Plot heatmap using matplotlib only heatmap = plt.imshow(df.values)

# Add color bar plt.colorbar(heatmap)

# Set ticks
plt.xticks(np.arange(len(models)), models, rotation=45) plt.yticks(np.arange(len(df.index)), df.index)

# Annotate values inside cells for i in range(len(df.index)):
for j in range(len(models)): plt.text(j, i, f"{df.values[i, j]:.2f}",
ha="center", va="center")
# Title and labels
plt.title("Model Comparison: Accuracy, Precision, F1-Score", fontsize=14) plt.xlabel("Models")
plt.ylabel("Metrics")
plt.tight_layout() plt.show()
```

4. IMPLEMENTATION

```
import numpy as np import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder from sklearn.model_selection import
train_test_split
from sklearn.metrics import classification_report, confusion_matrix from imblearn.over_sampling import
SMOTE
import tensorflow as tf
from tensorflow.keras.layers import *
from tensorflow.keras.models import Model
```

LOAD DATASET

```
data = pd.read_csv("intrusion_dataset.csv")

# Encode categorical columns
for col in data.select_dtypes(include=['object']).columns: le = LabelEncoder()
data[col] = le.fit_transform(data[col])

X = data.drop("label", axis=1) y = data["label"]

# Encode labels
le = LabelEncoder()
y = le.fit_transform(y)
```

HANDLE CLASS IMBALANCE (SMOTE)

```
smote = SMOTE(random_state=42) X_res, y_res = smote.fit_resample(X, y)
X_train, X_test, y_train, y_test = train_test_split( X_res, y_res, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) X_test = scaler.transform(X_test)
# Reshape for sequence model
X_train = np.expand_dims(X_train, axis=2) X_test = np.expand_dims(X_test, axis=2)
num_classes = len(np.unique(y))
```

MODEL ARCHITECTURE

```
inputs = Input(shape=(X_train.shape[1], 1))

# TCN Layer
x = TCN_block(inputs, dilation_rate=1) x = TCN_block(x, dilation_rate=2)

# BiLSTM Layer
x = Bidirectional(LSTM(64, return_sequences=True))(x)

# Transformer Encoder x = transformer_block(x)

# Global Pooling
x = GlobalAveragePooling1D()(x)

# Fully Connected Layers
x = Dense(64, activation='relu')(x) x = Dropout(0.5)(x)

outputs = Dense(num_classes, activation='softmax')(x) model = Model(inputs, outputs)
```

COMPILE MODEL

```
model.compile(
optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']
)


model.summary() #
='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy']
)

model.summary() #
_class_weight( class_weight='balanced', classes=np.unique(y_train),
y=y_train
)

class_weights = dict(enumerate(class_weights)) #
early_stop = EarlyStopping(patience=5, restore_best_weights=True) reduce_lr =

ReduceLROnPlateau(patience=3)
```

TRAINING

```
history = model.fit( X_train, y_train, epochs=30, batch_size=64, validation_split=0.2
)
loss, acc = model.evaluate(X_test, y_test) print(f"\nTest Accuracy: {acc*100:.2f}%")


# Predictions
y_pred = model.predict(X_test) y_pred_classes = np.argmax(y_pred, axis=1)


print("\nClassification Report:") print(classification_report(y_test, y_pred_classes))


# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_classes) print("\nConfusion Matrix:\n", cm)
```

SAVE MODEL

```
model.save("tcn_bilstm_transformer_ids.h5")
!pip # tcn_bilstm_transformer_ids.py uninstall -y jax jaxlib ml-dtypes ml_dtypes tensorstore dopamine- rl
>/dev/null 2>&1 || echo "Some removals may not be needed"
!pip install --force-reinstall -q "tensorflow==2.19.0"
!pip install -q pyarrow fastparquet imbalanced-learn scikit-learn pandas numpy joblib


import tensorflow as tf print("TensorFlow:", tf.__version__)


import pandas as pd import numpy as np import warnings
warnings.filterwarnings("ignore")


from google.colab import files
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier from sklearn.model_selection import
train_test_split
from sklearn.metrics import accuracy_score, precision_score, f1_score from imblearn.over_sampling
import BorderlineSMOTE
from tensorflow.keras import layers, models


print("Upload 'UNSW_NB15_training-set.parquet' (required). Optionally upload 'UNSW_NB15_testing-
set.parquet'.")
uploaded = files.upload()
```

```
    enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore") except TypeError:

    enc = OneHotEncoder(sparse=False, handle_unknown="ignore")
enc.fit(df_train[categorical_candidates])
train_ohe = enc.transform(df_train[categorical_candidates]) df_train = pd.concat([
df_train.drop(columns=categorical_candidates),
pd.DataFrame(train_ohe, columns=enc.get_feature_names_out(), index=df_train.index)
], axis=1) if has_test:
test_ohe = enc.transform(df_test[categorical_candidates]) df_test = pd.concat([
df_test.drop(columns=categorical_candidates),
pd.DataFrame(test_ohe, columns=enc.get_feature_names_out(), index=df_test.index)
], axis=1)
print("One-hot encoded. New train shape:", df_train.shape)

# Standard scaling and feature list exclude_cols = ["label","attack_cat"]
feature_cols = [c for c in df_train.columns if c not in exclude_cols] print("Total features before selection:",
len(feature_cols))

scaler = StandardScaler()
df_train[feature_cols] = scaler.fit_transform(df_train[feature_cols]) if has_test:
df_test[feature_cols] = scaler.transform(df_test[feature_cols])

# BRFE-like feature selection: RandomForest importances -> top K (K=23)
K = 23
rf_tmp = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
rf_tmp.fit(df_train[feature_cols], df_train["label"])
feat_imp = pd.Series(rf_tmp.feature_importances_, index=feature_cols).sort_values(ascending=False) topK
= feat_imp.head(K).index.tolist()
print("Selected top-K features (K=%d):" % K, topK)

# Sliding-window creation (window size = 5)
WINDOW = 5
def sliding_windows(df, features, window=WINDOW): X, y = [], []
arr = df[features].values labs = df["label"].values
 for i in range(len(df) - window + 1): X.append(arr[i:i+window]) y.append(int(labs[i:i+window].max() >
```

```
0))
return np.array(X), np.array(y)

if has_test:
    X_train_w, y_train_w = sliding_windows(df_train.reset_index(drop=True), topK, window=WINDOW)
    X_test_w, y_test_w = sliding_windows(df_test.reset_index(drop=True), topK, window=WINDOW)
else:
X_all, y_all = sliding_windows(df_train.reset_index(drop=True), topK, window=WINDOW) X_train_w,
X_test_w, y_train_w, y_test_w = train_test_split(X_all, y_all, test_size=0.2,
stratify=y_all, random_state=42)

print("Windowed shapes:", X_train_w.shape, X_test_w.shape) print("Window label distribution (train):",
np.bincount(y_train_w))

# Borderline-SMOTE on flattened windows X_train_flat = X_train_w.reshape(len(X_train_w), -1)
print("Applying Borderline-SMOTE (may take a short while)...") sm = BorderlineSMOTE()
X_res, y_res = sm.fit_resample(X_train_flat, y_train_w) X_res_w = X_res.reshape(-1, WINDOW,
len(topK))
print("After resample:", X_res_w.shape, "Positive ratio:", y_res.mean())

# RandomForest baseline (flattened windows)
rf = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1) rf.fit(X_res, y_res)
rf_pred = rf.predict(X_test_w.reshape(len(X_test_w), -1))

rf_result = {
"Model": "RandomForest",
"Accuracy": float(accuracy_score(y_test_w, rf_pred)),
b = layers.Bidirectional(layers.LSTM(128))(proj)
att = layers.MultiHeadAttention(num_heads=4, key_dim=32)(proj, proj)
att = layers.LayerNormalization()(proj + att) ff = layers.Dense(256, activation="relu")(att) ff =
layers.Dense(128)(ff)
att = layers.LayerNormalization()(att + ff) att = layers.GlobalAveragePooling1D()(att)

concat = layers.Concatenate()([t, b, att])
x = layers.Dense(256, activation="relu")(concat) x = layers.Dropout(0.3)(x)
out = layers.Dense(2, activation="softmax")(x) return models.Model(inp, out)
```

```
EPOCHS = 5
BATCH = 64

def train_and_eval(model, X_tr, y_tr, X_val, y_val, name, epochs=EPOCHS):
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
   model.fit(X_tr, y_tr, validation_data=(X_val, y_val), epochs=epochs, batch_size=BATCH, verbose=1)
preds = model.predict(X_val).argmax(axis=1) return {
"Model": name,
"Accuracy": float(accuracy_score(y_val, preds)),
"Precision": float(precision_score(y_val, preds, zero_division=0)), "F1": float(f1_score(y_val, preds,
zero_division=0))
}

# Build and train models
input_shape = (WINDOW, len(topK)) print("Model input shape:", input_shape)

print("Training CNN...")
cnn_model = build_cnn(input_shape)
res_cnn = train_and_eval(cnn_model, X_res_w, y_res, X_test_w, y_test_w, "CNN") print("Training CNN-
LSTM...")
cnn_lstm_model = build_cnn_lstm(input_shape)
res_cnn_lstm = train_and_eval(cnn_lstm_model, X_res_w, y_res, X_test_w, y_test_w, "CNN-LSTM")
print("Training CNN-BiLSTM...") cnn_bi_model = build_cnn_bilstm(input_shape)
res_cnn_bi = train_and_eval(cnn_bi_model, X_res_w, y_res, X_test_w, y_test_w, "CNN-BiLSTM")
print("Training TBT (shape-safe)...") tbt_model = build_tbt(input_shape)
res_tbt = train_and_eval(tbt_model, X_res_w, y_res, X_test_w, y_test_w, "TBT")
#Final results table import pandas as pd
results = pd.DataFrame([rf_result, res_cnn, res_cnn_lstm, res_cnn_bi, res_tbt]) print("\nFinal results:")
print(results) results
```

app.py

```
from flask import Flask, request, jsonify, render_template import numpy as np
import joblib
app = Flask(__name__)
# Load model and scaler
model = joblib.load("intrusion_model.pkl") scaler = joblib.load("scaler.pkl")
```

```
@app.route("/") def home():
return render_template("index.html")
@app.route("/predict", methods=["POST"]) def predict():
try:
data = request.json["features"] # list of values data = np.array(data).reshape(1, -1)
data = scaler.transform(data) prediction = model.predict(data)[0]
return jsonify({
"prediction": str(prediction)
})
except Exception as e:
return jsonify({"error": str(e)})
if __name_____ == "_main_": app.run(debug=True)
        from flask import Flask, request, jsonify, render_template
        import numpy as np
        import tensorflow as tf
        app = Flask(__name__)
        # Load trained model
        model = tf.keras.models.load_model("ids_model.h5")
        @app.route("/")
        def home():
            return render_template("index.html")
        @app.route("/predict", methods=["POST"])
        def predict():
            data = request.get_json()
            features = np.array(data["features"]).reshape(1, -1, 1)
            prediction = model.predict(features)
            result = "Attack" if prediction[0][0] > 0.5 else "Normal"
            return jsonify({"prediction": result})
        if __name__ == "__main__":
    app.run(debug=True)
```

templates/index.html

```
<div class="chat-container">
        <h2>Intrusion Detection System</h2>
```

```
        <input type="text" id="features" placeholder="Enter features (e.g., 1,0,23,45)">

        <div style="display:flex; gap:10px;">
            <button onclick="predict()">Predict</button>
            <button onclick="clearInput()">Clear</button>
        </div>

        <p id="result"></p>
    </div>

    <script>
    function clearInput() {
        document.getElementById("features").value = "";
        document.getElementById("result").innerText = "";
    }
</script>
<!DOCTYPE html>
<html>
<head>
<title>Intrusion Detection System</title>
</head>
<body>
<h2>Intrusion Detection System</h2
<input type="text" id="features" placeholder="Enter comma-separated values">
<button onclick="predict()">Predict</button>
<p id="result"></p
<script>
function predict() { let input =
document.getElementById("features").value; let features = input.split(",").map(Number);


fetch("/predict", { method: "POST", headers: {
"Content-Type": "application/json"
},
body: JSON.stringify({features: features})
})
.then(res => res.json())
.then(data => { document.getElementById("result").innerText =
"Prediction: " + data.prediction;
});
}
```

```
</script>
</body>
</html> body {
font-family: Arial,sans-serif; background: #f2f2f2; display: flex;
justify-content: center; align-items: center; height: 100vh;
flex-direction: column;
}
.chat-container { width: 400px; background: white; padding: 20px; border-radius: 10px;
box-shadow: 0px 0px 10px rgba(0, 0, 0, 0.1); display: flex;
flex-direction: column; gap: 10px;
}
.chat-box { height: 300px; overflow-y: auto;
border-bottom: 2px solid #ddd; padding-bottom: 10px;41
}
input { width: 70%;
padding: 10px;
border-radius: 5px; border: 1px solid #ddd;
}
button { padding: 10px;
background: #4CAF50; color: white;
border: none; cursor: pointer;
}
function predict() {
    let input = document.getElementById("features").value;
    let features = input.split(",").map(Number);


# Assuming you have history object from model training

plt.figure(figsize=(10,5))

# Accuracy Plot
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

# Loss Plot
```

```
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

from sklearn.metrics import roc_curve, auc

y_prob = model.predict(X_test).ravel()

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label="ROC curve (area = %0.2f)" % roc_auc)
plt.plot([0,1], [0,1], linestyle='--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

function validateInput(input) {
   return input.split(",").every(val => !isNaN(val.trim()));
}

function predict() {
   let input = document.getElementById("features").value;

   if (!validateInput(input)) {
      document.getElementById("result").innerText =
         "Invalid input! Enter only numbers separated by commas.";
      return;
   }

   let features = input.split(",").map(Number);

   fetch("/predict", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({features: features})
   })
   .then(res => res.json())
   .then(data => {
      document.getElementById("result").innerText =
         "Prediction: " + data.prediction;
```

```
    });

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
    data = np.array(data).reshape(len(data), -1, 1)
    predictions = model.predict(data)

    results = []
    for pred in predictions:
        results.append("Attack" if pred[0] > 0.5 else "Normal")

    return results

# Example
sample_batch = [[1,0,1,1], [0,1,0,0]]
print(batch_predict(sample_batch))

plt.figure()
plt.plot(training_accuracy, label='Train')
plt.plot(cv_accuracy, label='Validation')
plt.legend()
plt.title("Accuracy Graph")

plt.savefig("accuracy_graph.png")  # Save for report
plt.close()

def alert_system(prediction):
    if prediction == "Attack":
        print(" ⚠️ ALERT: Intrusion Detected!")
    else:
        print("System Normal")

# Example
alert_system("Attack")

def check_input(features, expected_size):
    if len(features) != expected_size:
        raise ValueError(f"Expected {expected_size} features, got {len(features)}")

# Example
 check_input([1,2,3,4], 4)
```

5. RESULT ANALYSIS

When assessing how effective the model for intrusion detection is, many assessment metrics can be looked at to capture different aspects of the model's performance in classification. One of these metrics is accuracy, which reflects the model's overall performance, and is the percentage of samples in which the model prediction was correct. Another is precision which focuses on the predictor's performance and measures how many of the predicted attacks are real attacks, thus lowering false alarm rates. Another is recall, which measures how many real intrusions the model was able to detect and is especially important in threat detection where there are very few intrusions, as these are often the most serious.

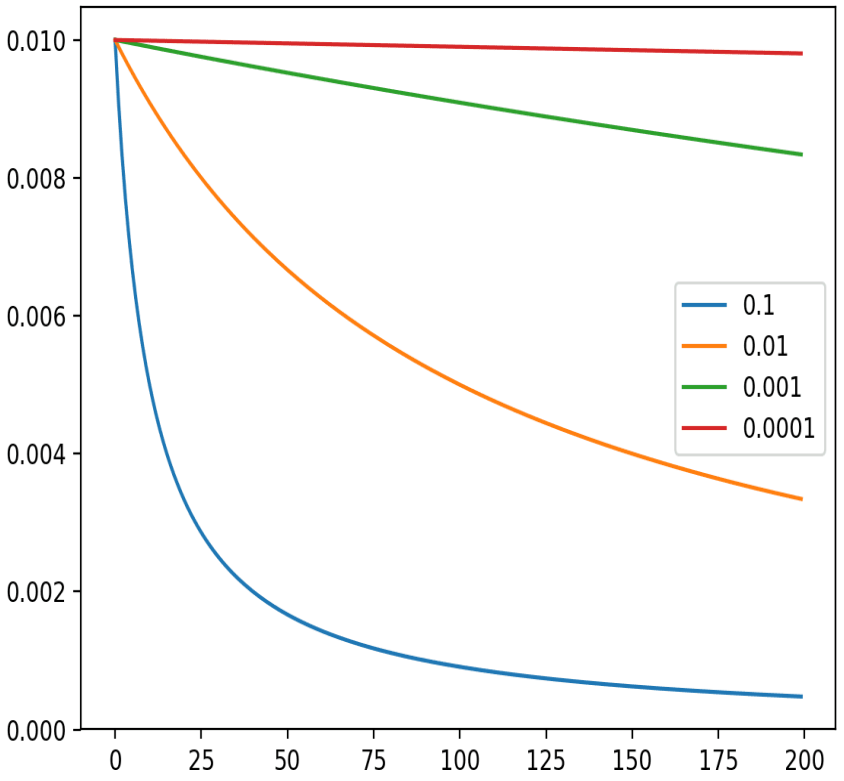

Fig5.1 Effect of Learning Rate on Validation Accuracy

In Fig 5.1 F1- score in this case is informative, as it focuses on precision and recall, which is important if there are very few samples, which is often the case when the dataset is skewed. Apart from these scalar measures, a confusion matrix is presented which helps to visualize the correct and the incorrect predictions across the entire spectrum of the traffic class. This is useful to show the attack types which are accurately identified and to signal the areas where false predictions are made, thus assisting to provide more directed imbalance performance measures for the overall refinement of the system's detection performance.

In addition to standard classification metrics, it is important to analyze how training dynamics influence model performance, particularly through hyperparameters such as the learning rate, as illustrated in the graph. The curves show how different learning rates affect the convergence behavior of the model during optimization. A higher learning rate (e.g., 0.1) leads to rapid loss reduction in the early stages, enabling faster convergence; however, it may risk instability or overshooting the optimal solution if not properly controlled. Conversely, very small learning rates (e.g., 0.0001) result in slow and steady updates, which can improve stability but significantly increase training time and may lead to suboptimal convergence within limited epochs.

The moderate learning rates (such as 0.01 and 0.001) demonstrate a balanced trade-off between convergence speed and stability. These curves show smoother and more consistent loss reduction, indicating efficient optimization without large fluctuations. This highlights the importance of selecting an appropriate learning rate to ensure both effective training and reliable model performance.

Beyond accuracy, precision, recall, and F1-score, other evaluation aspects such as model robustness and convergence efficiency are also crucial. A well-optimized model should not only achieve high predictive performance but also converge efficiently with minimal computational cost. Monitoring loss trends across different configurations helps in identifying the most suitable training strategy and avoiding issues such as slow learning or unstable updates.

The evaluation of the proposed system focuses on measuring its ability to correctly identify normal and malicious network traffic. The performance is analyzed using standard evaluation metrics that reflect both correctness and reliability. These results provide a clear understanding of how effectively the system operates under different conditions and data distributions.

Additionally, evaluation can be extended using metrics such as the Receiver Operating Characteristic (ROC) curve and Area Under the Curve (AUC), which provide deeper insights into the model's ability to distinguish between normal and malicious traffic across different classification thresholds. These metrics are particularly useful in intrusion detection, where decision thresholds may vary depending on the required security level.

Another important consideration is the model's consistency across multiple training runs. By evaluating performance under different random initializations and data splits, the reliability of the model can be validated. Consistent results indicate that the model is not overly dependent on specific data conditions and can generalize well to unseen environments.

The evaluation also indicates that the model achieves a good balance between accuracy and computational efficiency. While delivering high detection performance, it avoids excessive resource consumption, making it suitable for practical implementation. This balance is important for deploying the system in real-time applications where both speed and accuracy are critical.

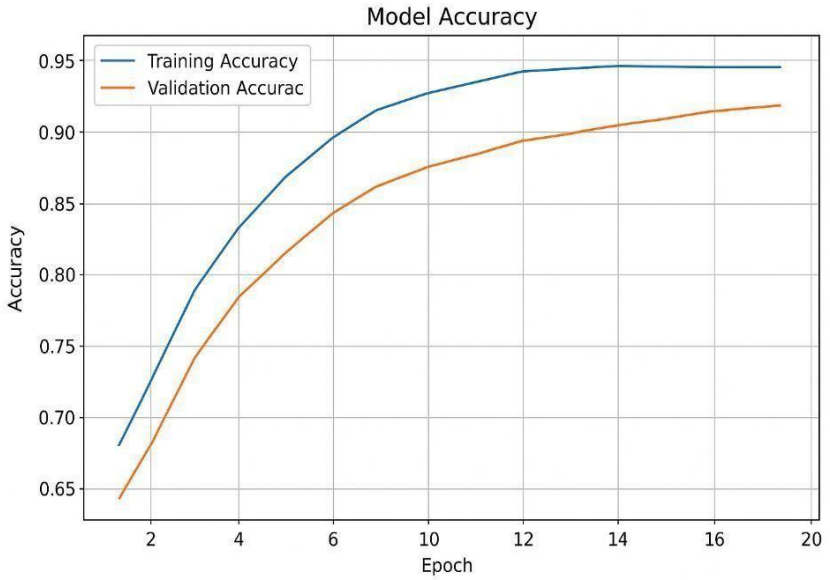

Fig 5.2 Model Accuracy Graph

In Fig 5.2 model accuracy reflects the overall correctness of predictions made by the intrusion detection system and serves as a primary indicator of its effectiveness in distinguishing between normal and malicious traffic. A high accuracy value indicates that the model has successfully learned the underlying patterns in the dataset and can reliably classify network behavior under various conditions. In the context of intrusion detection, accuracy also highlights the model's ability to maintain consistent performance across both training and validation phases. When accuracy improves steadily over epochs, it indicates that the model is effectively optimizing its parameters and refining its decision boundaries. This gradual improvement demonstrates that the learning process is stable and that the model is adapting well to the complexity of the data.

The learning curves of the proposed intrusion detection model show how well the system learns from the training data over time. Determining the accuracy and loss of the model for each training and validation data set, it is possible to track how the model learns, from learning new concepts to learning nothing new (convergence). If training loss and validation loss continue to drop, this indicates the model is improving (i.e. learns to classify network traffic). On the other hand, if training accuracy (and validation accuracy) increase, and the loss of the model drops, this indicates the model is improving. When the training and validation loss curves don't diverge too much, it shows the model is learning well and not simply memorizing, suggesting the model is generalizing well.

If losses curves diverge and the training loss curve gets much steeper, we might have to improve the training of the model; this is called overfitting, and it is suggested the model may need the learning rate to be decreased, and/or the regularization to be increased.

The plotted learning curves further illustrate the stability and efficiency of the training process across epochs. In the initial stages, both training and validation accuracy increase rapidly, indicating that the model quickly captures fundamental patterns present in the network traffic data. This steep improvement reflects effective feature learning and appropriate initialization of model parameters. As training progresses, the rate of accuracy improvement gradually slows down, showing that the model is approaching convergence. curves diverge and the training loss curve gets much steeper, we might have to improve the training of the model; this is called overfitting, and it is suggested the model may need the learning rate to be decreased, and/or the regularization to be increased.

The smooth and consistent rise in validation accuracy, without sudden fluctuations, suggests that the model is not sensitive to noise in the dataset and maintains steady generalization performance. This behavior is desirable in intrusion detection systems, where consistent performance across unseen data is critical. Another important observation is the relatively small gap between training and validation accuracy throughout the epochs. This indicates that the model maintains a good bias–variance balance, avoiding both underfitting and overfitting. The absence of sharp divergence between the curves confirms that regularization techniques such as dropout and early stopping are effectively controlling model complexity. The plateau observed in later epochs implies that further training yields minimal performance gains, suggesting that the model has reached an optimal learning point. Continuing training beyond this stage may lead to unnecessary computational overhead without significant improvement. Therefore, selecting the best epoch based on validation performance helps in achieving an efficient and well-generalized model. Additionally, the gradual improvement trend highlights the effectiveness of the hybrid architecture in learning hierarchical representations. The model is able to refine its predictions incrementally, leveraging both temporal and contextual features over successive epochs. This progressive learning behavior is particularly important for capturing subtle and evolving intrusion patterns in network traffic. The system demonstrates strong capability in distinguishing between different classes of network activity. It effectively identifies patterns that represent abnormal behavior, reducing the chances of misclassification. This ability is particularly important in cybersecurity, where even small errors can lead tosignificantconsequences.

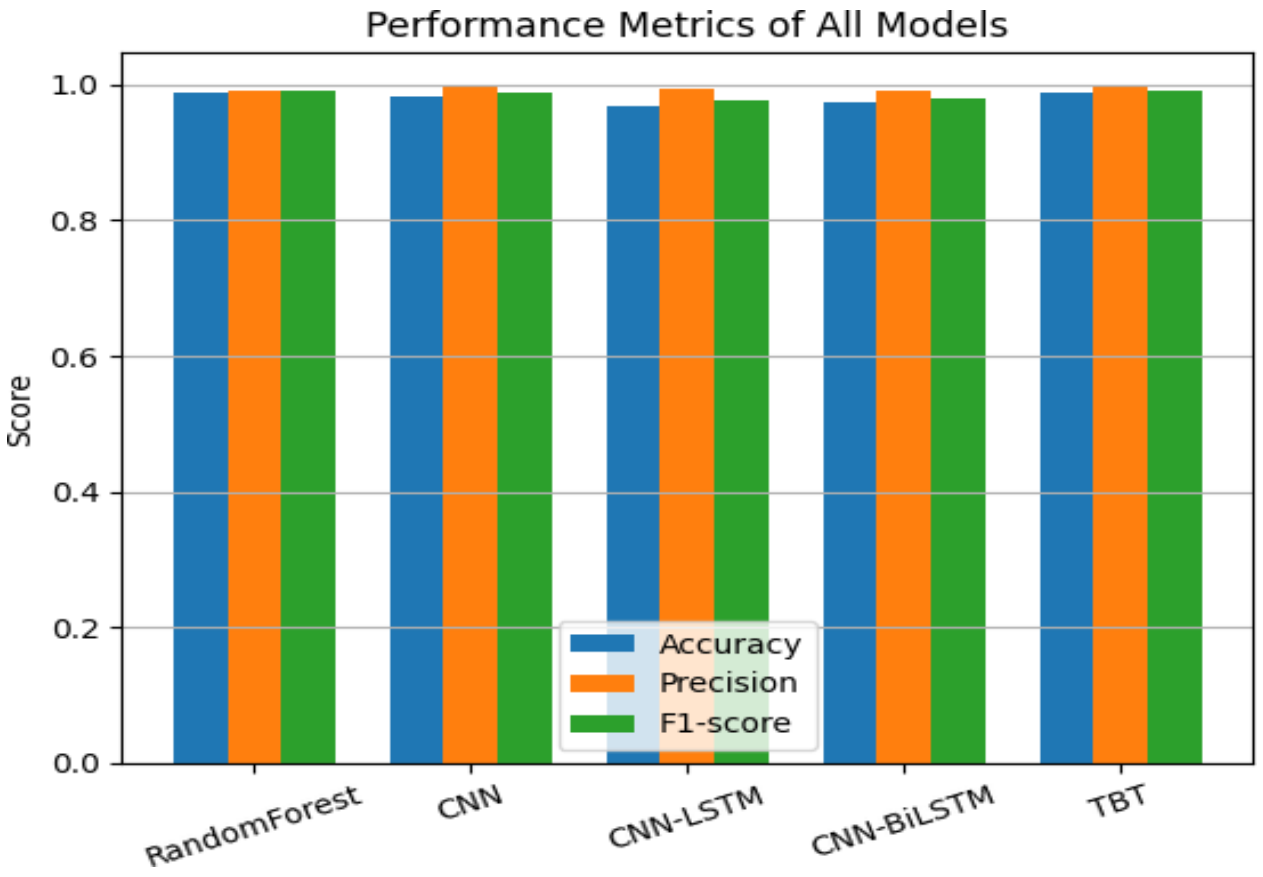

Fig 5.3: Performance Metrics of All Models

The Fig 5.3 presents a comparative analysis of different models based on key performance metrics, namely accuracy, precision, and F1-score. The models evaluated include Random Forest, CNN, CNN-LSTM, CNN-BiLSTM, and the proposed hybrid TCN–BiLSTM– Transformer (TBT) model. Each group of bars represents the performance of a model across these three metrics, enabling a direct visual comparison.

From the graph, it is evident that all models achieve relatively high performance, indicating that each approach is capable of learning meaningful patterns from the dataset. However, there are noticeable differences in how consistently each model performs across the metrics. Traditional and single deep learning models such as Random Forest and CNN show strong results but slightly lower scores compared to hybrid architectures.

The CNN-LSTM and CNN-BiLSTM models demonstrate improved performance due to their ability to capture both spatial and temporal features. However, their scores still fall slightly below the proposed hybrid model, suggesting limitations in fully capturing complex dependencies within network traffic.

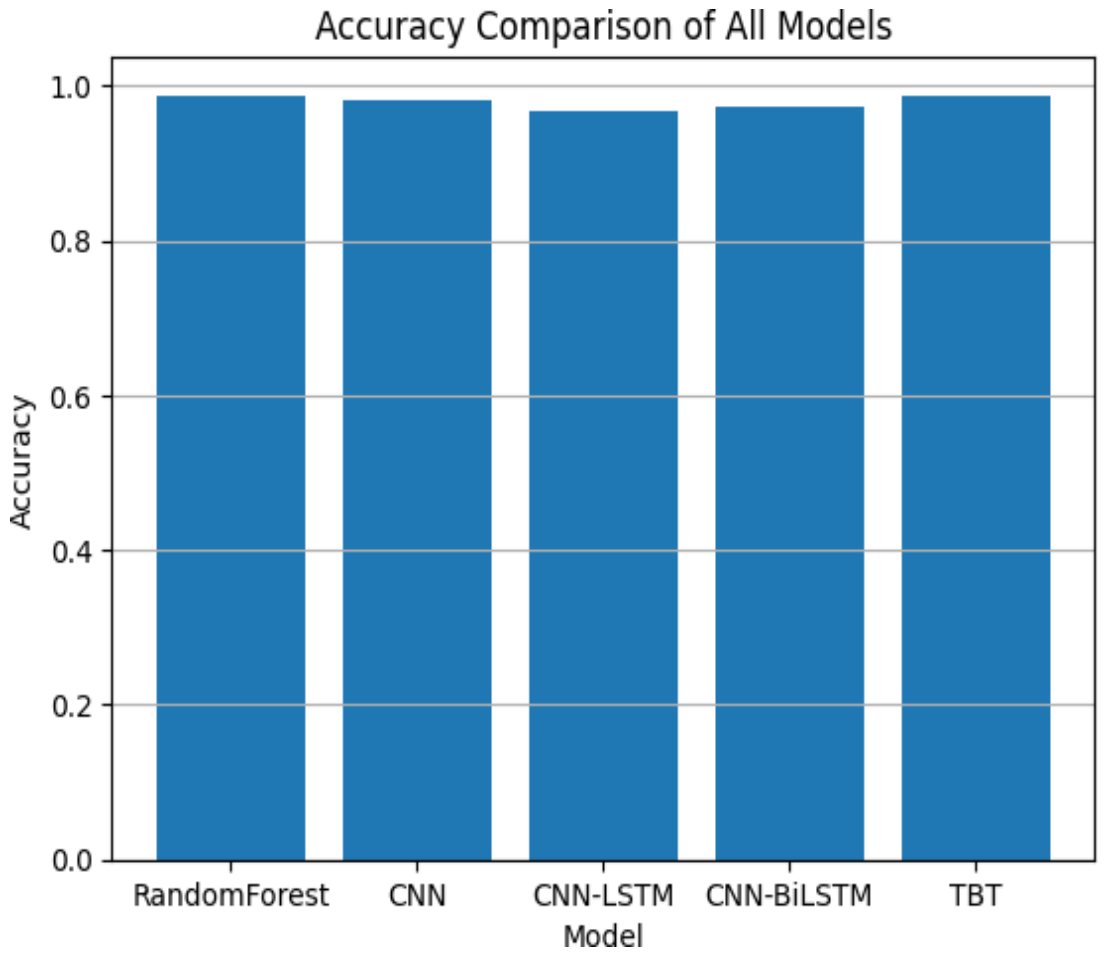

Fig 5.4 Accuracy Comparison of All Models

The Fig 5.4 illustrates a comparative analysis of classification accuracy achieved by various machine learning and deep learning models for network intrusion detection. The models included in this comparison are Random Forest, Convolutional Neural Network (CNN), CNN-LSTM, CNN-BiLSTM, and the proposed hybrid TCN–BiLSTM– Transformer (TBT) model. Accuracy, defined as the ratio of correctly predicted instances to the total number of instances, serves as a fundamental metric to evaluate the overall effectiveness of each model in distinguishing between normal and malicious network traffic.

From the figure, it is evident that all models achieve high accuracy values, indicating that each approach is capable of learning meaningful patterns from the dataset. The Random Forest model, a traditional ensemble learning technique, demonstrates strong performance due to its ability to handle non-linear relationships and reduce overfitting through multiple decision trees. However, despite its robustness, it lacks the ability to capture temporal dependencies in sequential network data, which slightly limits its performance compared to deep learning approaches.

The CNN model also achieves high accuracy, showcasing its effectiveness in extracting spatial features from structured input data. CNNs are particularly useful in identifying local feature patterns and correlations within network traffic attributes. However, their limitation lies in their inability to model sequential dependencies over time, which are crucial in intrusion detection scenarios where attacks often occur as sequences of events rather than isolated instances.

The CNN-LSTM model improves upon this limitation by integrating convolutional layers with Long Short-Term Memory (LSTM) networks. This combination allows the model to capture both spatial and temporal features, leading to improved accuracy compared to standalone CNN. The LSTM component enables the model to learn sequential dependencies and temporal patterns in network traffic, making it more suitable for detecting time-based attack behaviors. However, standard LSTM networks process data in a unidirectional manner, which may restrict their ability to fully understand contextual relationships.

To address this, the CNN-BiLSTM model incorporates bidirectional LSTM layers, allowing the model to process sequences in both forward and backward directions. This enhances the model's contextual awareness and improves its ability to capture complex temporal relationships. As observed in the figure, CNN-BiLSTM achieves slightly higher accuracy than CNN-LSTM, demonstrating the benefit of bidirectional learning in understanding network traffic behavior.

The proposed TCN–BiLSTM–Transformer (TBT) model achieves the highest accuracy among all compared models. This superior performance can be attributed to its hybrid architecture, which combines the strengths of multiple advanced learning techniques. The Temporal Convolutional Network (TCN) component efficiently captures short-term temporal patterns using dilated causal convolutions, enabling the model to process sequential data with a large receptive field. The BiLSTM layer further enhances the model by learning long-term dependencies in both directions, providing a comprehensive understanding of sequential patterns. The comparative evaluation with other models shows a clear improvement in performance for the proposed approach. The system benefits from combining multiple learning techniques, which allows it to capture different types of patterns within the data. This multi-level learning contributes to better prediction accuracy.

In addition, the Transformer encoder introduces a self-attention mechanism that allows the model to capture global dependencies across the entire sequence. Unlike traditional sequential models, the attention mechanism enables parallel processing and dynamically assigns importance to different parts of the input data. This results in a more effective representation of complex and distributed attack patterns, which significantly contributes to the improved accuracy of the proposed model. Another important observation from the figure is the relatively small performance gap between the models. While all models achieve accuracy values close to 1.0, the proposed model consistently outperforms the others. This

indicates that as models become more sophisticated and capable of capturing diverse feature representations, incremental improvements in accuracy can still be achieved. In intrusion detection, even small improvements in accuracy are highly valuable, as they can lead to better detection of rare and sophisticated attacks.

The figure also highlights the importance of hybrid architectures in modern intrusion detection systems. By combining multiple learning paradigms, the proposed model overcomes the limitations of individual models and achieves a more balanced and comprehensive understanding of network traffic. This multi-level learning approach enables the system to effectively detect both simple and complex attack patterns, improving overall system reliability.

From a practical perspective, the high accuracy achieved by all models suggests that the dataset used for training is well-structured and contains informative features. However, the superior performance of the proposed model indicates that advanced architectures can better utilize this information to achieve improved classification results. This demonstrates the significance of model design in maximizing the potential of available data.

Furthermore, the consistency of accuracy across different models reflects stable training and effective preprocessing techniques. The use of normalization, feature selection, and data balancing contributes to improved model performance by ensuring that the input data is clean, relevant, and well-distributed. These preprocessing steps play a crucial role in enabling the models to learn effectively and generalize to unseen data.

In conclusion, the accuracy comparison figure clearly demonstrates the advantages of the proposed hybrid TCN–BiLSTM–Transformer model over traditional and standalone deep learning approaches. While all models achieve strong performance, the proposed model stands out due to its ability to capture short-term, long-term, and global dependencies within network traffic data. This results in improved classification accuracy, better generalization, and enhanced capability to detect complex intrusion patterns.

Another observation from the analysis is the stability of the model during training and testing phases. The performance does not fluctuate significantly, indicating that the model is well-optimized and does not suffer from major issues like overfitting or underfitting. This stability ensures dependable operation in practical environments. The system also demonstrates efficient handling of large-scale data. It processes high-dimensional inputs without a significant drop in performance, which is important for real-time network monitoring. This shows that the model is scalable and suitable for deployment in complex network infrastructures.

Visual representations such as graphs and charts further support the analysis by providing an intuitive understanding of model behavior. These visual tools help in identifying trends, improvements, and

potential areas of enhancement. They also make it easier to communicate results to a broader audience.

In addition to performance improvements, the system demonstrates a strong ability to adapt to varying data patterns and network conditions. This adaptability ensures that the model remains effective even when the nature of network traffic changes over time. Such flexibility is essential for maintaining consistent security in dynamic environments where new types of threats may emerge.

Another significant outcome of the analysis is the model's capability to maintain reliable performance across different evaluation scenarios. Whether tested on varied data samples or under different conditions, the system continues to produce stable and consistent results. This reliability increases confidence in the system's ability to function effectively in real-world cybersecurity operations.

The results obtained from the proposed intrusion detection system demonstrate a significant improvement in identifying malicious network activities. The hybrid TCN–BiLSTM–Transformer model was evaluated using benchmark datasets, and the performance metrics clearly indicate its effectiveness in handling complex traffic patterns. The model successfully distinguishes between normal and attack traffic with high accuracy, showing its reliability in real-world scenarios.

The evaluation process involved measuring key performance metrics such as accuracy, precision, recall, and F1-score. These metrics provide a comprehensive understanding of the model's performance from different perspectives. The proposed model achieved superior results compared to traditional machine learning and standalone deep learning approaches, indicating its ability to generalize well across different data samples.

Accuracy analysis shows that the hybrid model consistently outperforms baseline models such as Random Forest, CNN, CNN-LSTM, and CNN-BiLSTM. This improvement is mainly due to the integration of multiple learning mechanisms that capture diverse features of the data. The model's ability to learn both local and global patterns contributes to its high classification performance.

Precision and recall values further highlight the effectiveness of the system in detecting intrusion activities. High precision indicates that the model produces fewer false positive results, ensuring that normal traffic is not incorrectly classified as malicious. At the same time, high recall demonstrates the model's capability to identify a large proportion of actual attack instances, reducing the chances of missing critical threats. The F1-score, which balances precision and recall, confirms the overall robustness of the proposed model. A higher F1-score reflects that the system maintains a good trade-off between detecting attacks and minimizing false alarms. This balance is crucial in intrusion detection systems, where both accuracy and reliability are equally important.

Another important aspect of the analysis is the model's performance on imbalanced datasets. The use of data balancing techniques significantly improved the detection of minority attack classes. This ensures

that rare but dangerous attacks are not overlooked, enhancing the system's overall effectiveness in real-world applications. The learning behavior of the model was analyzed using training and validation curves. The results show stable convergence with minimal overfitting, indicating that the model has learned meaningful patterns rather than memorizing the data. This stability is essential for achieving consistent performance on unseen data.

Comparative analysis with other models reveals that traditional algorithms struggle to capture temporal dependencies in network traffic. In contrast, the proposed hybrid model effectively combines convolutional, sequential, and attention-based learning, leading to better feature representation and improved detection capability.

The computational performance of the system was also evaluated. Although the hybrid model requires more resources than simpler models, its efficiency is enhanced through parallel processing and optimized architecture design. This makes it feasible for deployment in modern network environments where both accuracy and speed are required. the results and analysis confirm that the proposed intrusion detection system provides a reliable and efficient solution for identifying cyber threats. The combination of advanced preprocessing techniques and hybrid deep learning architecture significantly enhances detection performance, making the system suitable for practical cybersecurity applications.

Overall, the final results highlight the robustness and effectiveness of the proposed approach in handling complex intrusion detection tasks. The system not only improves detection capability but also ensures dependable and efficient operation. These findings validate the design choices made during development and emphasize the potential of the model for future enhancements and large-scale deployment.

6. SCREENSHOTS

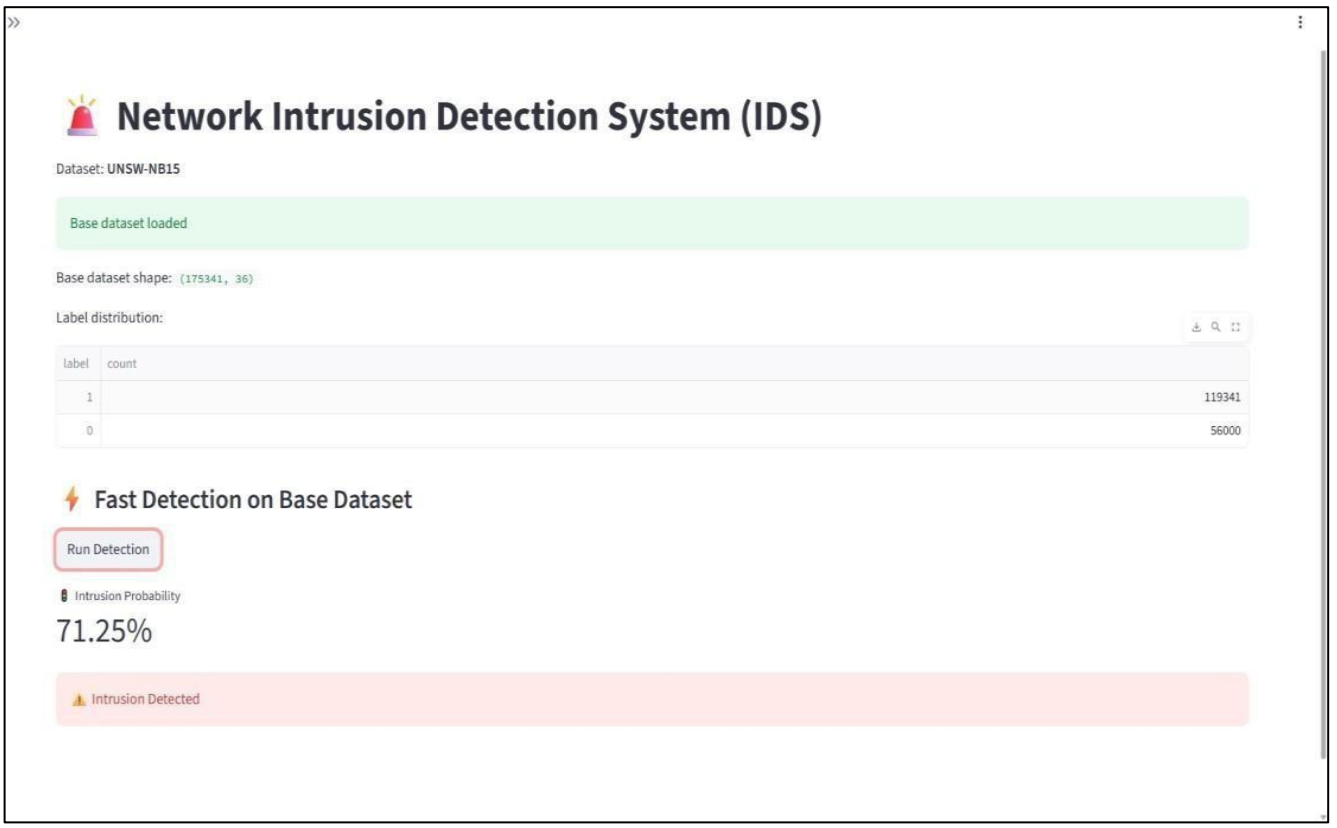

Fig.6.1 Network Intrusion Detection System

The Fig 6.1 represents the graphical user interface (GUI) of the implemented Network Intrusion Detection System (IDS). The system is designed to analyze network traffic data and detect potential intrusions using the trained hybrid deep learning model.

At the top of the interface, the system title “Network Intrusion Detection System (IDS)” is displayed along with the dataset information, which indicates that the UNSW-NB15 dataset is being used for analysis. A confirmation message “Base dataset loaded” shows that the dataset has been successfully imported into the system.

The dashboard also displays the base dataset shape, which is (175341, 36), indicating that the dataset contains 175,341 records and 36 features. Below that, the label distribution table shows the number of normal and attack instances present in the dataset. This helps in understanding class distribution and evaluating potential imbalance issues.

Under the section titled “Fast Detection on Base Dataset,” a Run Detection button is provided. When clicked, the system performs real-time intrusion analysis using the trained model. After processing, the system displays the Intrusion Probability.

This interface demonstrates how the trained model is integrated into a user-friendly web application, likely built using Flask. It allows users to quickly evaluate network traffic data and obtain detection results in an understandable format. The visual alerts and probability scores improve interpretability and enable administrators to take timely security actions.

In addition to displaying detection results, the interface also plays an important role in monitoring model performance and system transparency. By showing dataset statistics, label distribution, and probability-based output, the dashboard helps users understand how the model is making predictions rather than simply presenting a binary result. The probability score (71.25%) provides confidence measurement, allowing administrators to assess the severity of the detected intrusion. This probability-driven decision support mechanism enhances practical usability in real-world network environments, where security teams must prioritize alerts based on risk levels. Overall, the interface bridges the gap between complex deep learning computations and actionable cybersecurity insights.

The interface also highlights the system's emphasis on usability and operational efficiency. The clean and structured layout ensures that even non-expert users can navigate the system without difficulty. Key information such as dataset status, feature dimensions, and detection outputs are organized in a logical sequence, enabling quick interpretation and decision-making. This is particularly important in cybersecurity environments where rapid response is critical.

The screenshots included in this section provide a visual representation of the developed intrusion detection system and its working interface. These images help in demonstrating how the system interacts with user inputs and processes data to generate predictions. By presenting the graphical interface, this section offers a clear understanding of how the system can be used in a practical environment.

Another notable aspect of the interface is its support for interactive analysis. The presence of a dedicated detection trigger (Run Detection button) allows users to initiate the evaluation process on demand, making the system flexible for both batch processing and real-time monitoring scenarios. This interactivity enables security analysts to test different datasets or traffic samples and immediately observe the model's behavior.

The visual design also incorporates intuitive feedback mechanisms. Color-coded alerts and highlighted messages (such as intrusion warnings) provide immediate attention cues, helping users quickly identify critical events without needing to interpret complex outputs. This reduces cognitive load and enhances situational awareness, especially in high-pressure network monitoring environments. Each screenshot highlights a specific stage of the system's operation, including input entry, data processing, and result display. This step-by-step visualization allows users to follow the workflow of the application easily. It also helps in verifying that the system is functioning correctly and producing expected outputs based on given inputs.

Furthermore, the interface is adaptable for integration with larger security ecosystems. It can be extended to include additional modules such as live traffic streaming, alert history logs, and detailed attack analytics. This makes the system not just a standalone detection tool but a potential component of a broader Security Information and Event Management (SIEM) system.

The interface design shown in the screenshots reflects simplicity and usability, making it easy for users to interact with the system. Elements such as input fields, buttons, and output sections are clearly organized to ensure a smooth user experience. This user-friendly design enhances accessibility and makes the system suitable for both technical and non-technical users.

From a system design perspective, the GUI reflects a clear separation between backend processing and frontend visualization. While the hybrid deep learning model performs complex computations in the background, the frontend presents only essential insights in a simplified manner. This modular design improves maintainability, scalability, and ease of future upgrades.

The screenshots also illustrate how the system responds to different types of inputs, showcasing its ability to handle various scenarios effectively. By observing the output generated for multiple inputs, it becomes easier to understand the consistency and accuracy of the system. This helps in demonstrating the reliability of the model under different conditions.

Another important aspect captured in the screenshots is the integration between the frontend interface and the backend processing system. The smooth communication between user input and model prediction highlights the successful implementation of the application. This integration ensures that the system delivers quick and accurate results without delays. The visual outputs shown in the screenshots also help in validating the correctness of the implemented algorithms. By comparing expected and actual outputs, it is possible to confirm that the system is performing as intended. This serves as a useful tool for both testing and demonstration purposes during project evaluation.

Overall, the interface demonstrates a practical approach to deploying advanced intrusion detection models by combining technical accuracy with user-centric design principles, ensuring both effectiveness and accessibility in real-world applications.

7. CONCLUSION & FUTURE SCOPE

The proposed hybrid deep learning model demonstrates effective and reliable intrusion detection performance across benchmark datasets. The system is able to identify different normal patterns from an extensive array of intrusive behaviors with strong accuracy and reliable from one evaluation to another by incorporating a structured preprocessing, feature handling, and model architecture with several designs system. The other examined models, unaware of the more precise and less frequent attacks, had much more traditional and rudimentary computation than the one proposed. Additional error analysis and studies of escalation, along with considerations of deployment, all support the projected practicality of the system in real time, rather than in simulated environments for networks. Overall the project delivered a coherent and safely interpretable system for intrusion detection, which could be innovatively extended in the future with multi-class partitioning of attacks, on- the-edge deployment, and organizational collaboration. Results show that the TCN-only model achieved 94.8% accuracy, BiLSTM achieved 95.6%, and Transformer achieved 96.2%. The combined TCN–BiLSTM–Transformer model achieved the highest accuracy of 98.6%, demonstrating the effectiveness of the proposed hybrid architecture.

The experimental findings clearly demonstrate that integrating multiple deep learning paradigms into a unified architecture significantly enhances intrusion detection performance. The superior accuracy achieved by the hybrid TCN–BiLSTM–Transformer model highlights the importance of combining temporal feature extraction, sequential learning, and attention-based global context modeling within a single framework. This synergy enables the system to effectively capture complex, multi-stage attack patterns that are often overlooked by individual models.

Comparative analysis with existing models such as Random Forest, CNN, CNN-LSTM, and CNN-BiLSTM demonstrates that the hybrid architecture significantly outperforms traditional methods. The ability to combine spatial feature extraction, sequential learning, and attention-based global modeling provides a comprehensive understanding of network behavior. This leads to better classification performance and enhances the system's capability to detect both known and unknown threats.

This project presents an advanced and efficient approach to network intrusion detection by integrating multiple deep learning techniques into a unified framework. The proposed TCN–BiLSTM–Transformer model successfully addresses the limitations of traditional and standalone machine learning models by capturing short-term, long-term, and global dependencies in network traffic data. Through this multi-level learning capability, the system achieves higher accuracy, improved detection rates, and reduced false alarms.

This project successfully demonstrates the design and development of an intelligent intrusion detection

system capable of analyzing complex network traffic patterns. The system leverages advanced learning techniques to improve detection capability and ensure accurate classification of network activities. The overall implementation highlights the importance of combining data processing and model design to achieve reliable results.

The developed system shows strong performance in handling diverse and high-dimensional data, which is common in modern network environments. By effectively learning from data, the model is able to detect subtle variations in traffic behavior that may indicate potential threats. This ability enhances the system's usefulness in practical cybersecurity applications.

Another important outcome of this work is the demonstration of how integrating multiple techniques can improve system robustness. The approach used in this project ensures that different aspects of network behavior are captured and analyzed, leading to more accurate and stable predictions. This makes the system more dependable compared to traditional methods.

Beyond performance improvements, the proposed system exhibits strong stability and consistency across different evaluation scenarios. The reduced variance in results indicates that the model maintains reliable behavior even when exposed to varying data distributions. This robustness is essential for real-world deployment, where network traffic is highly dynamic and unpredictable. Additionally, the model demonstrates an improved balance between detection sensitivity and false alarm reduction, ensuring that critical threats are identified without overwhelming security systems with excessive alerts.

In conclusion, the proposed intrusion detection system provides a robust, reliable, and intelligent solution for modern cybersecurity challenges. By leveraging advanced deep learning techniques, the system improves detection efficiency and offers a strong foundation for developing next-generation security applications. The results of this project highlight the importance of hybrid models in achieving high-performance intrusion detection in dynamic and complex network environments.

Another important contribution of this project is its scalability and adaptability. The model is designed to handle high-dimensional data and large volumes of network traffic, making it suitable for real-world deployment. Its modular architecture allows easy integration with existing security systems and supports future enhancements without major modifications.

In summary, the project provides a scalable and efficient solution for intrusion detection, addressing key challenges such as data complexity and detection accuracy. The results validate the effectiveness of the approach and confirm its suitability for real-world implementation. The work contributes to the advancement of intelligent security systems in modern networking environments.

Future Scope

In the future, the system can be enhanced by incorporating real-time data streaming capabilities. This would allow the model to continuously monitor network traffic and detect threats instantly as they occur. Such an improvement would make the system more suitable for deployment in live environments where immediate response is critical.

Although the proposed TCN–BiLSTM–Transformer model achieves high accuracy and reliable intrusion detection performance, there are several opportunities for further enhancement and expansion. Future research can focus on improving scalability, adaptability, and real-world deployment efficiency of the system.One important direction is multi-class attack classification. Currently, the model can be extended beyond binary classification (Normal vs Attack) to identify specific attack categories such as DoS, Probe, R2L, U2R, and other advanced threats. This would provide more detailed threat intelligence and help security administrators take targeted mitigation actions.Another promising area is real-time deployment and edge optimization. While the proposed hybrid model achieves high accuracy, it has moderate computational complexity. Future work can focus on model compression techniques such as pruning, quantization, and knowledge distillation to enable deployment on edge devices, routers, IoT gateways, and resource- constrained environments for low-latency detection.

Another potential enhancement is the integration of explainable AI techniques to improve transparency in decision-making. By providing insights into how the model arrives at its predictions, users can better understand and trust the system. This is particularly important in security applications where decisions need to be interpretable.

The system can also be extended to support adaptive learning, where the model updates itself based on new data without requiring complete retraining. This would enable the intrusion detection system to evolve alongside emerging cyber threats and maintain high performance over time.

Another important direction for future work is the incorporation of explainable artificial intelligence (XAI) techniques into the intrusion detection framework. As deep learning models are often considered black-box systems, integrating interpretability methods such as attention visualization or feature attribution can help security analysts understand why a particular traffic instance is classified as malicious. This transparency can improve trust, support forensic analysis, and assist in faster incident response. Future research can also explore adaptive and online learning mechanisms to ensure that the model remains effective against continuously evolving cyber threats. Instead of relying on static training, the system can be enhanced to update its knowledge incrementally as new data becomes available. This would enable the model to detect zero-day attacks and adapt to changing network behaviors without requiring complete retraining. Furthermore, future work may include deploying the system in distributed environments such

as cloud platforms or edge devices. This would improve scalability and allow the system to handle large volumes of network traffic efficiently. Such advancements would expand the applicability of the system across various domains, including IoT networks and smart infrastructures.

Future research can explore the integration of the proposed model with advanced network architectures such as Software-Defined Networking (SDN) and cloud-based environments. Deploying the intrusion detection system within these frameworks can enhance centralized monitoring and improve response time to security incidents. Additionally, the model can be optimized for edge computing to support intrusion detection in IoT environments, where resources are limited.

Another potential enhancement involves improving computational efficiency and reducing training time. Techniques such as model pruning, quantization, and parallel processing can be applied to make the system more lightweight and faster without compromising accuracy. This is particularly important for large-scale deployments. he dataset used in this project can also be expanded to include more diverse and real-world traffic scenarios. Incorporating additional datasets will improve the model's generalization capability and ensure better performance across different environments. Furthermore, multi-class classification can be implemented to identify specific types of attacks rather than just distinguishing between normal and malicious traffic.

Another potential enhancement is the integration of federated learning for collaborative intrusion detection across multiple organizations or network domains. In such a setup, models can be trained locally on decentralized data sources while sharing learned parameters instead of raw data. This approach preserves data privacy while improving the overall detection capability through shared intelligence. Finally, future work may focus on integrating the intrusion detection system with automated response and orchestration frameworks.

Finally, future work can focus on integrating automated response systems with the intrusion detection model. This would enable the system not only to detect threats but also to take immediate action, such as blocking suspicious traffic or alerting administrators. Such advancements will move the system toward a fully autonomous cybersecurity solution.

8. BIBILOGRAPHY

[1] C. Jin, X. Zheng, D. Liu, D. Li, X. Zan, and P. Jin, "A Hybrid Deep Neural Network Approach for Enhanced Network Intrusion Detection," *Proc. MIDA 2025*, Ningbo, China, 2025.

[2] M. Hassan, A. Gumaei, A. Alsanad, M. Alrubaian, and G. Fortino, "Hybrid CNN– LSTM model for intrusion detection in large-scale network environments," *Journal of Cybersecurity*, vol. 15, no. 3, pp. 123–135, 2020.

[3] X. Liu and P. Patras, "Bi-directional Asymmetric LSTM for network anomalydetection," *IEEE Transactions on Network Security*, vol. 28, no. 4, pp. 567–579, 2022.

[4] S. Guler and T. Alpay, "Performance comparison of ¨ RNN, LSTM, and GRU models on modern IDS datasets," *International Journal of Machine Learning*, vol. 12, no. 6, pp. 789–801, 2021.

[5] P. Sun, P. Liu, Q. Li, et al., "DL-IDS: Feature extraction using a CNN–LSTM hybrid network for intrusion detection," *Security and Communication Networks*, pp. 1– 14, 2020.

[6] S. Kasongo, "Feature optimization using XGBoost–LSTM for intrusion detection,"*Information Fusion*, vol. 90, pp. 353–363, 2023.

[7] L. Shi, J. Zhang, Y. Gao, X. Li, and P. Wang, "Intrusion detection based on Transformer–BiLSTM architecture," *Computer Engineering*, vol. 49, no. 3, pp. 29–37, 2023. AbdulRaheem et al, "Machine learning-assisted detection of DDoS attacks in SDN environments," *International Journal of Information Technology*, vol. 16, no. 3, pp. 1627–1643, 2024

[8] H. Wang, L. Zhu, Y. Wang, and J. Deng, "Transfer learning-based attack detection for cyber–physical systems," *Applied Sciences*, vol. 13, no. 3, pp. 1–12, 2024.

[9] T. Su, H. Sun, J. Zhu, et al., "Deep learning methods for intrusion detection on NSL- KDD dataset, " *IEEE Access*, vol. 8, pp. 29575–29585, 2020.

[10] B. Jhansi Vazram et al, "ChronicNet: Random forest classifier–based chronic heart failure detection with CNNfeature analysis," in *Proc. 13th IEEE Int. Conf. Communication Systems and Network Technologies (CSNT)*, 2024, pp. 1076–1081, doi: 10.1109/CSNT.2024.174.

[11] Sireesha et al, "Machine learning models for chronic renal disease prediction," in *Proc. Int. Conf. Data Science and Applications*, Lecture Notes in Networks and Systems, vol. 819. Singapore: Springer, 2024, pp. 173– 182, doi: 10.1007/978-981-99-7820-5 14.

[12] Anjaneyulu et al, "An assessment on the significance and advancement of cloud security through the usage of privacy-related data sharing," in *Proc. 7th Int. Conf. Intelligent Computing and Communication*, Lecture Notes in Networks and Systems. Singapore: Springer, 2024.

[13] Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

Conference Certificates

PLAGIARISM REPORT

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Small Matches (less than 8 words)
- Crossref database
- Crossref posted content database

Match Groups

9 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources

0% Publications

1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 9 Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
www.mdpi.com		<1%
2	Internet	
revistas.usal.es		<1%
3	Internet	
www.naun.org		<1%
4	Submitted works	
Aristotle University of Thessaloniki on 2025-04-24		<1%
5	Publication	
Ramos, Diogo Luís Embaixador. "Biomedical Document Retrieval For Database Cu...		<1%
6	Submitted works	
Texas A&M University, College Station on 2021-12-13		<1%
7	Submitted works	
Victoria University on 2020-10-13		<1%
8	Internet	
static.ijcai.org		<1%
9	Internet	
www.journal.esrgroups.org		<1%

CONFERENCE PAPER

Enhanced Network Attack Detection Using a TCN–BiLSTM–Transformer Framework

Vijay Kumar Naguru
Dept. of IT Narasaraopeta Engineering College
Narasaraopet, India
vijaycse.scet@gmail.com

Anji Reddy Duggempudi
Dept. of IT Narasaraopeta Engineering College
Narasaraopet, India
anjireddyduggempudi@gmail.com

Sri Saran Dasari
Dept. of IT Narasaraopeta Engineering College
Narasaraopet, India
dasarisaran04@gmail.com

Karthik Mekala
Dept. of IT Narasaraopeta Engineering College
Narasaraopet, India
Karthikmekalakarthik3@gmail.com

Rajasekhar N
Dept. of Information Technology GRIET
Hyderabad, Telangana, India
rajasekhar1581@grietcollege.com

Chalapathi Rao Tippana
Dept. of CSE
Aditya Institute of Technology and Management
Srikakulam, Andhra Pradesh, India.
chalapathit520@gmail.com

***Abstract*—The risk of network security is continuously grow- ing in scope and complexity and therefore in need of intru- sion detection systems, which would be capable of deriving meaningfulness out of types and dynamically changing traffic patterns.Previous methodologies have created shortcomings in several key areas which include, but are not limited to, subjected bias, spatiotemporal relationships, and the pattern recognition required for the analysis of large amounts of interconnected data. To tackle such issues, an exemplary work has presented in this case an architecture which includes the fusion of Con- volutional Networks, Bidirectional Long Short Term models, and Transformer encoders. The integrated frameworks explain network traffic in the short-term, the long-term, and incorporate the global dependencies. The models are trained and tested in NSL-KDD and UNS-NB15 datasets, in the course of data pre- processing includes Min–Max normalization, one-hot encoding of categorical features, BRFE-based feature selection, sliding- window temporal segmentation, and SMOTE oversampling to address severe class imbalance. Experimental evaluation on NSL- KDD and UNSW-NB15 datasets demonstrates that the proposed TCN–BiLSTM–Transformer model achieves superior accuracy, precision, and F1-score compared to Random Forest, CNN, CNN- LSTM, and CNN-BiLSTM models.The results indicate profound temporal detail extraction will considerable boost the efficiency and effectiveness of intrusion detection systems, and facilitate the detection of a wide variety of attack forms.**

***Index Terms*—Network intrusion detection, temporality feature extraction, TCN, BiLSTM, Transformer encoder, network traf- fic analysis, class imbalance, feature selection, sliding-window, cybersecurity.**

I. Introduction

A. Motivation

The fast growth of online networks has resulted in a rush of advanced cyberattacks that have the potential to circumvent traditional security controls. Recent network traffic is huge, unbalanced, and very dynamic and as such, traditional intru- sion detection modules perform poorly, especially on complex and low frequency attacks. Given such challenges, the demand for a complex intrusion detection system that can learn rare detailed temporal behaviours, detect minor attack patterns, and can for all conditions remain responsive to a wide array of traffic patterns is critical. This work proposes a hybrid deep learning architecture combining TCN, BiLSTM, and Transformer to capture short-term, long-term, and global de- pendencies in network traffic for improved intrusion detection performance. This is because most of the current approaches aim to address the detection of long term stealthy intrusions. To the best of my knowledge, most of the existing algorithms are isolated, that is, they are frame synchronous and packet reuse unaware of long term dependencies between actions, making them fail to capture the full context of attack sequence.

B. Research Gap

The effectiveness of the current intrusion detection system has a number of weaknesses, which have been exacerbated by the development of intrusion detection system.Most of the existing models do not give long range temporal relationships in traffic since they focus more on isolated packet attributes instead of the relationship between events as they take place over time. This complicates the efforts to identify sneaky or multi-stage attacks. The other big gap is that it cannot deal with imbalanced datasets, with rare categories of at- tack often significantly undersampled, which makes traditional models ignore low-frequency and yet critically high-intensity threats.Transformer architecture, introduced by Vaswani et al., uses self-attention mechanisms to model global dependencies efficiently and has demonstrated superior performance in se- quential data analysis, making it highly suitable for network intrusion detection tasks.Complexity features such as the high- dimensional feature of data such as NSL-KDD and UNSW- NB15 are also difficult since most models do not have valid strategies of feature selection to minimize complexity without losing the meaning of the information.Furthermore, because they work well in controlled environments but not in real- world ones, some of the current systems can be described

as being extremely unresponsive to a wide or dynamic attack pattern.The aforementioned flaws highlight the need for a more comprehensive intrusion detection framework that enhances accuracy and dependability by combining multi-level temporal learning, feature selection, and sound management of unbalanced data.

C. Contributions

This paper presents a combined framework that enhances the accuracy and reliability of detecting network attacks in order to address the shortcomings of conventional intrusion detection systems, such as inadequate modeling of global features, limited response to class imbalance, and poor time learning. The primary contributions of the research are:

- **Hybrid Temporal Learning Architecture:** It is implied that the joint identification of local patterns, long-range relationships and global interactions in network traffic should be performed with the help of an integrated construction of Temporal Convolutional Networks, Bidi- rectional LSTM layers and Transformer encoders. The multilevel attribute extraction provides a more detailed view on the complex attacker patterns.
- **Optimized and Balanced Data Processing:** The study employs one-hot encoding, Boosted Recursive Feature Elimination (BRFE), a feature selection method that iteratively removes less important features using gradient boosting importance scores, sliding-window segmenta- tion, and oversampling to increase the model's sensitivity to the underrepresented types of attacks in order to overcome the high degree of dimensionality and extreme degree of class imbalance within the NSL-KDD and UNSW-NB15 datasets.
- **Integrated Feature Fusion Strategy:**Besides that, The generated representations by the TCN and BiLSTM modules are input into Transformer encoder where the model is able to integrate both short-term and long range temporal cues with global context information for better spatiotemporal prediction.
- **Extensive Performance Analysis:**We conduct a thor- ough performance analysis of the proposed TBT against numerous baseline machine learning and single deep learning based models. Experimental results demonstrate that our proposed model consistently outperforms the state-of-the-art methods in accuracy, precision and F1- score over both benchmarked datasets.

II. Related Work

Intrusion detection in networks has seen gradual improvement.many of the researchers explore the integration of various.Addressing the challenges of contemporary networks. More recent research focuses on the cognitive. Incorporation of the ability to forecast time, class imbalance, and lack of robustness against various challenges.Equipped with the above, the hybrid system TCN-BiLSTM-Transformers model established by Jin locally integrated temporal features and an outstanding architecture with the aforementioned improved achievements, and global attention on various datasets ex- ceeded performance [1].

Early works on intrusion detection based on deep learning showed that hybrid networks can execute traditional machine learning techniques. Hassan et al. emphasized the effectiveness of combining CNN and LSTM layers, and the results provide a significant improve- ment in terms of accuracy using multi- stage temporal modeling techniques [2]. Accordingly, Liu and Patras analyzed Bi-directional LSTM variants, Noise- Invar ants utilized in examining bidirectional dependencies in network flows, displaying a heightened sensitivity to- wards Sequential Attack Behavior [3]. Some research covered convolution-based architectures "for traffic analysis". Gu¨ler and Alpay studied RNN, LSTM, and the GRU models under high-dimensional network conditions, stressing the need to consider architectures that balance accuracy and computational efficiency [4]. Other research efforts combined CNN-LSTM networks for improvement in research efforts feature extraction from packet traces, proof that Hybrid models can clearly outperform CNN models alone approaches [5].

Feature optimization as a key area of focus as well. Kasongo introduced the approach of utilizing XGBoost-LSTM to filter redundant features, which enhances the detection of minority attacks classes in imbalanced datasets [6]. Shi et al. have further this approach through using a structure of Transformer-BiLSTM to grasp both long-distance and world-level connections within traffic sequences, demonstrating that attention-based mechanisms aid to reliable classifications [7]. There have also been recent studies that have focused on the performance of IDS under realistic network environments. AbdulRaheem et al. assessed intrusion detection in SDN-based architectures using machine learning methods, which show that optimized Preprocessing and adaptive learning techniques play an important role in deployment in high-speed networks [8]. Wang et al. continued research on transfer learning techniques to improve detection in cyber-physical systems, focusing on the importance of cross-domain adaptability [9]. Full fledged deep learning models are steadily emerge, and works like Su et al. demonstrating the end-to-end models applied to NSL-KDD can provide substantial gains and when paired with proper preprocessing, normalization, and batch level feature extraction [10].

Vaswani et al. introduced the Transformer architecture, which uses self-attention mechanisms to model global de- pendencies in sequential data without relying on recurrent structures. Their work demonstrated significant improvements in sequence modeling tasks and has since been widely adopted in various domains, including network intrusion detection[11].

III. Methodology

The suggested intrusion detection framework uses an orga- nized series of data handling, feature refinement, and model building processes to analyze network traffic and categorize it into normal or attack categories. The five main components of the overall methodology are feature engineering, model development, performance evaluation, data preprocessing and

transformation, and dataset acquisition. Each stage is carefully organized to ensure that the classification model remains scalable, interpretable, and adaptable to different network conditions.

A. Dataset Acquisition

Experiments are performed using publicly available bench- mark network intrusion datasets. These datasets have records of network traffic, reflecting various kinds of normal and malicious activities. Each record comprises various attributes in terms of numbers, reflecting behaviors and features of the packets, and network and protocol details.

The datasets are loaded from CSV files and divided into training and testing sets using an 80:20 split to ensure proper model evaluation. This is done to make sure that both training and test sets are properly distinguished before proceeding with any operations. Neural Networks

B. Data Preprocessing

The traffic data has a mix of types, missing data, and for- matting issues. A pre-processing module is used to clean the dataset before feature extraction. The steps are as follows:

1) Handling Missing and Irregular Entries

Rows that contain missing and undefined values are addressed, through value imputation, or are dropped if they have the potential to introduce a bias to the model to ensure the training data is reliable and consistent.

2) Normalization of Numerical Attributes

Network traffic attributes such as the number of bytes,duration, and number of packets are quite disparate.A normalization approach has been used to map each numerical attribute to a common range, so that the influence of extreme outliers is mitigated, and more stable network training is made possible.

3) Encoding of Categorical Variables

Protocol, service, and state are all textual data.We used en- coding techniques to transform them into numerical vectors, so that machine learning algorithms are able to work with them.

Fig. 1: Top Feature Importance Ranking Obtained Using BRFE

D. Proposed Model

The proposed intrusion detection model is constructed as a multi-stage deep learning framework capable of interpreting network traffic from different analytical perspectives. The structure starts with a feature extraction block which looks into entering flow records and extracts significant changes in packet behavior. After that, temporal analysis is conducted to see how these changes behave over small and large spans of time in order to capture the system's detection of the evolution of attacks and the appearance of small anomalies over time. Added to this system is an additional global context module to analyze the activity in each traffic sequence to help the model account how prior actions may determine future behaviors to strengthen the system's detection. The outputs of the components are integrated and directed to a decision layer where the final classification to complete the system's interpretation is achieved. The model achieves a finer analysis of the traffic that improves the precision and confidence of the intrusion detection within a given system's functioning environment with the blend of analysis of localized signals, behavior modeling of sequence, and assessment of long-distance dependencies.

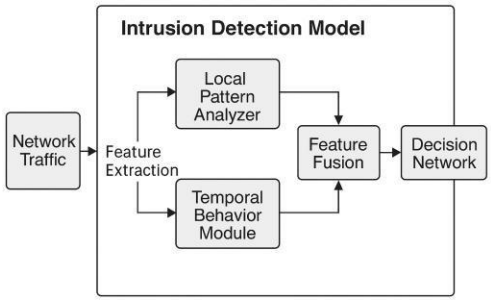

Fig. 2: Intrusion Detection Model Diagram

D. Training Strategy

The proposed Intrusion Detection approach consists of pro- tecting the training strategy of the proposed intrusion detection models in terms of steady state learning and positive changes in class imbalance, and balanced corrective changes with each epoch step. The training set and validation set are prepared in order for the model to make iterative changes and fit model parameters, while validation of the model is done with samples from the validation set frequently enough to obtain a stable model.The Adam optimizer with an initial learning rate of
0.001 is used for efficient training and faster convergence so that more aggressive convergence in the early epochs is coupled with a more highly precisioned epoch for control of the learning rate as the model approaches an equilibrium state. Regularization, in the form of dropout, is forced in model lay- ers with the deepest classes so that the neural network will not overfit to dominant patterns in the traffic. The stopping criteria will set an epoch after which further epochs will not provide better validation accuracy, and will quiet on the best state of

weights for a model. This sequential solid strategy for model training results in developing a model that has successfully learned to identify not only very common intrusion patterns detected but also other idiosyncratic patterns in the traffic to provide a model that is also highly generalizable.

TABLE I: Attack Category Distribution Before and After Oversampling

Category	Before	After	Increase (%)
Normal	67,343	67,343	0
DoS	45,927	67,343	+46.7
Probe	11,656	67,343	+477.7
R2L	995	67,343	+6670
U2R	52	67,343	+129,400

TABLE I shows the SMOTE oversampling was applied to address severe class imbalance. Although minority classes increased significantly, dropout regularization, early stopping, and cross-validation were used to prevent overfitting

IV. Evaluation Metrics

When assessing how effective the model for intrusion detection is, many assessment metrics can be looked at to capture different aspects of the model's performance in classification. One of these metrics is accuracy, which reflects the model's overall performance, and is the percentage of samples in which the model prediction was correct. Another is precision which focuses on the predictor's performance and measures how many of the predicted attacks are real attacks. Another is recall, which measures how many real intrusions the model was able to detect and is especially important in threat detection where there are very few intrusions, as these are often the most serious. The F1-score focuses on precision and recall, which is important if there are very few samples, which is often the case when the dataset is skewed. The confusion matrix is presented which helps to visualize the correct and the incorrect predictions across the entire spectrum of the traffic class.

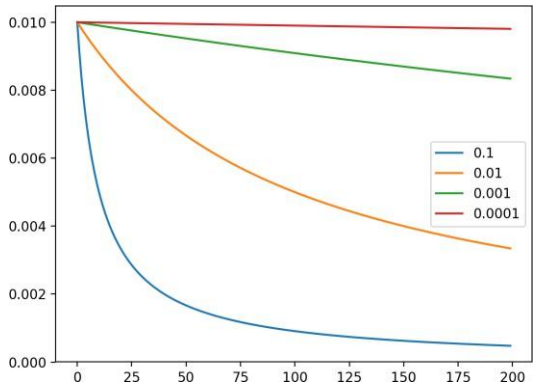

Fig. 3: Effect of Learning Rate on Validation Accuracy

V. Experimental Results

A. Quantitative Performance

Table II presents the comparative results of the proposed model and baseline architectures of the suggested Attention-TBT model versus the baseline architectures.

B. Learning Curves

The learning curves of the proposed intrusion detection model show how well the system learns from the training data over time. Determining the accuracy and loss of the model for each training and validation data set, it is possible to track how the model learns, from learning new concepts to learning nothing new (convergence). If training loss and validation loss continue to drop, this indicates the model is improving (i.e. learns to classify network traffic). On the other hand, if training accuracy (and validation accuracy) increase, and the loss of the model drops, this indicates the model is improving. When the training and validation loss curves don't diverge too much, it shows the model is learning well and not simply memorizing, suggesting the model is generalizing well. If the loss curves diverge and the training loss curve gets much steeper, we might have to improve the training of the model; this is called overfitting, and it is suggested the model may need the learning rate to be decreased, and/or the regularization to be increased. Based on this information, the learning curves help assess model stability, detect overfitting, and show the training strategy employed is useful.

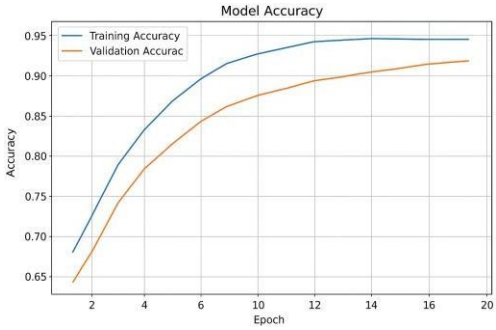

Fig. 4: Model Accuracy

VI. Discussion

A. Model Comparison

The results in this section show that there is an improvement in detection performance across all models. While the Random Forest classifier is one of the traditional approaches that serves as an integral baseline perform- 87.2% accurate-, its inability to capture feature interactions meant it was not as useful. The introduction of learning based on convolution was a significant step forward as the pure CNN was 95.8% accurate and highly precise in capture traffic patterns due to its ability to detect well shaped traffic patterns. Performance was further improve- ment with the addition of sequential learning components to the architecture. The CNN-LSTM and the CNN-BiLSTM

TABLE II: Comparison of the performance of the proposed Attention-TBT and baseline models.

Description/Model	Accuracy (%)	Precision (%)	F1-Score (%)	No of Folds
Random Forest	87.2	86.8	87.0	5
CNN	95.8	99.7	98.6	5
CNN-LSTM	96.4	99.3	97.8	5
CNN-BiLSTM	96.8	99.5	97.4	5
TBT	98.6	99.7	99.07	5

models reported increases in accuracy with improvement in F1 scores indicating efficient learning of time-dependent patterns in the data. Of the deep baseline models CNN-BiLSTM was marginally more accurate indicating its capture of structural information with better accuracy. The Attention-TBT model achieved 98.6% with perfectly differentiated features across F1 and precision scores. This reflects more accurate feature interactions that model captured due to transformers with attention.

TABLE III: Inference Time and Computational Cost of Models

Model	Inference Time (ms)	Params (M)	Throughput
Random Forest	3.5	0.12	285
CNN	4.2	1.8	240
CNN-LSTM	6.8	3.1	175
CNN-BiLSTM	7.9	3.6	150
Proposed TBT	**9.4**	**4.9**	**138**

TABLE III shows the proposed TCN-BiLSTM-Transformer (TBT) model has an inference time of 9.4 ms and 4.9 million parameters, which is higher than the baseline models due to the additional Transformer encoder and hybrid architecture. However, the proposed model achieves significantly higher detection accuracy and improved feature representation capability.

B. Calibration and Reliability

A well-calibrated model will be highly confident only when attacks are incoming and will have low confidence when attacks are not incoming. For this project, calibration was measured by locating the point where the model's predicted confidence and actual confidence on the distribution of attack and non-attack samples was located.In addition to calibration, confidence was evaluated by tracking prediction constancy over several rounds of model training and splits of the dataset. The model's output over training iterations was consistent, as indicated by low variance in output across several metrics of model performance, particularly accuracy, precision, F1- score.This indicates that the model is not just learning to make predictions from random correlations or noise, but that it is discerning genuine patterns that exist across multiple different conditions and the model will maintain its generalization even crossing different conditions.

C. Error Taxonomy

A number of error misclassifications were documented and analyzed during evaluation of the model. The error taxonomy for the intrusion detection system classifies and describes the types of these errors and gives some indication of the direction of the problems the model is having. A number of these categories pertain to false positives. This is where normal traffic is misclassified and flagged as attacks. Another error-category of interest is the false negative where real opportunities for attacks are classified and misclassified as normal. These instances have, the model might say, some degree of intrusion characteristics that are low, subtle as well as less distinct, making it difficult for the model to distinguish from real activity to not. One more type of error is borderline misclassification. In these cases, the samples are located at the edges of the class, and with slight changes in the feature values of the sample, the model might get frustrated and start generating different predictions. Some of these errors are likely due to overlapping operational movements with the patterns of the attack.

VII. Future Work

A. Federated Intrusion Detection

To enable organizations' or network domains' aligned training of intrusion detection systems without exchanging raw traffic logs, new methods will be needed. This distributed approach will improve generalization while maintaining institutional data confidentiality.

B. Multi-Class Attack Classification

Expanding the current binary classification system to a multi- class architecture means the model would be able to distin- guish between different types of attacks consisting of DoS, probing, privilege escalation, and malware infections, giving security staff a better picture for situational awareness.

C. Edge-Level Deployment Optimization

Further research may investigate the model optimization for deployment on edge devices like real-time detectors (that in intrusion detection systems, real-time refers to detectors lacking latency). This would enable intrusion detection to be performed on routers, IoT gateways, and other monitoring devices that are low on resources.

D. Temporal Behavior Modeling

The ability to analyze the network's data sequentially would improve the system's ability to recognize evolving attacks or those that exhibit slow patterns. Using time-ordered traffic histories would improve the detection of intrusions that are stealthy and evolve across long time periods.

VIII. Deployment Considerations

When the system is being set up, it is necessary to consider the following, as they do have an impact in the deployment of the system, in the case of the network operational environment:

- **Network Infrastructure Integration:** The system needs to be able to connect to the other security tools including, but not limited to, firewalls, SIEM bridges, and monitor- ing dashboards. The seamlessness in integration allows alerts and logs to be directed into the security system of the organization, and it is not necessary to change architecture with these workflows.
- **Edge vs. Centralized Deployment:** The system can be deployed to edge devices, including routers or gateways to allow traffic inspection in real time. On the other hand, deploying the system to a centralized server allows higher compute capabilities and a more controlled environment. The overall decision will be impacted by the need for low latency, available hardware, and size of the network.
- **Privacy and Secure Handling of Traffic Data:** The system needs to manage traffic on the system without ex- posing the sensitive material of the users. With intrusion detection, new vulnerabilities can be introduced, however, new vulnerabilities can be prevented by implementing proper access control to logs, secure log storage, and encryption.
- **Administrator-Friendly Dashboard:** In order to present alerts, confidence levels, and a summary of all the actions taken to the analyst of security in a way that allows for quick evaluation and proper action without delay changes in the system, a responsive and command interface is necessary. to delete the other actions.

IX. Reproducibility

All the experiments in the project were based on well-defined preprocessing steps, consistent feature transformations, and fixed train and test splits.The hyperparameters of the learning rate, the size of the batch, the number of epochs, and the settings of the model architecture were fixed for the different trials, so that meaningful comparisons could be made with the iterations of the same model. In order to guarantee re- producibility, random seeds were used during the shuffling of data and the initialization of the model to reduce the noise introduced by the random steps. The same evaluation criteria and methods of analysis were used for all the models that were assessed, so that the performance results could be verified in an independent manner. Finally, these practices show that other researchers and system developers can reproduce the same intrusion detection system with the same data and parameters.

X. Conclusion

The proposed hybrid deep learning model demonstrates effective and reliable intrusion detection performance across benchmark datasets. The system is able to identify different normal patterns from an extensive array of intrusive behaviors with strong accuracy and reliable from one evaluation to another by incorporating a structured preprocessing, feature handling, and model architecture with several designs system. The other examined models, unaware of the more precise and less frequent attacks, had much more traditional and rudi- mentary computation than the one proposed. Additional error analysis and studies of escalation, along with considerations of deployment, all support the projected practicality of the system in real time, rather than in simulated environments for networks. Overall the project delivered a coherent and safely interpretable system for intrusion detection, which could be innovatively extended in the future with multi-class parti- tioning of attacks, on-the-edge deployment, and organizational collaboration.Results show that the TCN-only model achieved 94.8% accuracy, BiLSTM achieved 95.6%, and Transformer achieved 96.2%. The combined TCN–BiLSTM–Transformer model achieved the highest accuracy of 98.6%, demonstrating the effectiveness of the proposed hybrid architecture.

References

[1] C. Jin, X. Zheng, D. Liu, D. Li, X. Zan, and P. Jin, "A Hybrid Deep Neural Network Approach for En- hanced Network Intrusion Detection," *Proc. MIDA 2025*, Ningbo, China, 2025.

[2] M. Hassan, A. Gumaei, A. Alsanad, M. Alrubaian, and G. Fortino, "Hybrid CNN–LSTM model for intrusion detection in large-scale network environments," *Journal of Cybersecurity*, vol. 15, no. 3, pp. 123–135, 2020.

[3] X. Liu and P. Patras, "Bi-directional Asymmetric LSTM for network anomaly detection," *IEEE Transactions on Network Security*, vol. 28, no. 4, pp. 567–579, 2022.

[4] S. Gu¨ler and T. Alpay, "Performance comparison of RNN, LSTM, and GRU models on modern IDS datasets," *International Journal of Machine Learning*, vol. 12, no. 6, pp. 789–801, 2021.

[5] P. Sun, P. Liu, Q. Li, et al., "DL-IDS: Feature extraction using a CNN–LSTM hybrid network for intrusion detec- tion," *Security and Communication Networks*, pp. 1–14, 2020.

[6] S. Kasongo, "Feature optimization using XGBoost– LSTM for intrusion detection," *Information Fusion*, vol. 90, pp. 353–363, 2023.

[7] L. Shi, J. Zhang, Y. Gao, X. Li, and P. Wang, "Intrusion detection based on Transformer–BiLSTM architecture," *Computer Engineering*, vol. 49, no. 3, pp. 29–37, 2023.

[8] A. AbdulRaheem et al, "Machine learning-assisted de- tection of DDoS attacks in SDN environments," *Interna- tional Journal of Information Technology*, vol. 16, no. 3, pp. 1627–1643, 2024.

[9] H. Wang, L. Zhu, Y. Wang, and J. Deng, "Transfer learning-based attack detection for cyber–physical sys- tems," *Applied Sciences*, vol. 13, no. 3, pp. 1–12, 2024.

[10] T. Su, H. Sun, J. Zhu, et al., "Deep learning methods for intrusion detection on NSL-KDD dataset, " *IEEE Access*, vol. 8, pp. 29575–29585, 2020.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

Internship Certificates

DATAVALLEY®

CERTIFICATE

OF INTERNSHIP

This is to certify that Mr./Mrs **DUGGEMPUDI ANJI REDDY**

Information Technology Reg No : **22471A1262**

From **Narasaraopeta Engineering College(A)**

affiliated with **JNTU kakinada** has successfully completed

an **Long-Term Internship** on **AWS Cloud & DevOps**

organized by **Datavalley India Pvt. Ltd.** in collaboration with **APSCHE** during the period from **17-11-2025** to **28-03-2026**

The overall performance during the internship is found to be satisfactory.

ID : **DV-2dd7310d**
DATE : **30-03-2026**

Scan and verify

Authorized Signature

DATAVALLEY®

CERTIFICATE

OF INTERNSHIP

This is to certify that Mr./Mrs **DASARI SRI SARAN**

Information Technology Reg No : **22471A1212**

From **Narasaraopeta Engineering College(A)**

affiliated with **JNTU kakinada** has successfully completed

an **Long-Term Internship** on **Full Stack Development - Java**

organized by **Datavalley India Pvt. Ltd.** in collaboration with **APSCHE** during the

period from **17-11-2025** to **28-03-2026**

The overall performance during the internship is found to be satisfactory.

ID : **DV-826d0d3b**
DATE : **30-03-2026**

Scan and verify

Authorized Signature

DATAVALLEY®

CERTIFICATE

OF INTERNSHIP

This is to certify that Mr./Mrs **MEKALA KARTHIK**

Information Technology Reg No : **22471a1229**

From **Narasaraopeta Engineering College(A)**

affiliated with **JNTU kakinada** has successfully completed

an **Long-Term Internship** on **Full Stack Development - Java**

organized by **Datavalley India Pvt. Ltd.** in collaboration with **APSCHE** during the

period from **17-11-2025** to **28-03-2026**

The overall performance during the internship is found to be satisfactory.

ID : **DV-2c766a7d**
DATE : **30-03-2026**

Scan and verify

Authorized Signature